

Web Veri Madenciliği – Kapsamlı Sınav Hazırlık Raporu

Yıldız Teknik Üniversitesi – Bilgisayar Mühendisliği Bölümü

Haziran 2026

İçindekiler

1 Birliktelik Kuralları Madenciliği ve Apriori Algoritması	2
1.1 Pazar Sepeti Modeli ve Temel Kavramlar	2
1.2 Apriori Algoritması	2
1.2.1 Apriori Özelliği ve Monotonluk	2
1.2.2 Seviye-Seviye Arama ve Aday Üretimi	3
1.2.3 Sık Ürün Kümelerinden Kural Üretimi	3
1.3 Sayma Yöntemleri: Üçgensel Matris ve Üçlüler	4
1.4 PCY Algoritması	4
1.5 Multistage Algoritması	4
1.6 Multihash Algoritması	5
1.7 Sıralı Örüntü Madenciliği (Sequential Pattern Mining)	5
1.8 Sınıf Birliktelik Kuralları (CAR)	6
1.9 Veri Formatı Dönüşümleri	6
2 Bilgi Erişimi ve Metin İşleme	7
2.1 Giriş ve DIKW Hiyerarşisi	7
2.2 Bilgi Erişimi ve Veritabanı Erişimi Karşılaştırması	7
2.3 Boolean Modeli	7
2.4 Vektör Uzay Modeli ve TF-IDF	8
2.4.1 TF (Terim Frekansı) Ağırlıklandırması	8
2.4.2 IDF (Ters Doküman Frekansı)	8
2.4.3 TF-IDF Birleşik Ağırlığı	8
2.4.4 Kosinüs Benzerliği ile Erişim	8
2.5 Okapi BM25	9
2.6 Rocchio İlgililik Geri Bildirimi	9
2.7 Metin Ön İşleme	10
2.7.1 Durma Kelimelerinin Çıkarılması	10
2.7.2 Gövdeleme	10
2.8 Ters İndeks Yapısı	10
2.9 Değerlendirme: Kesinlik ve Geri Çağırma	11
2.10 IR ve Web Araması İlişkisi	11
3 Web Bağlantı Analizi, PageRank, HITS ve Merkezilik Ölçütleri	13
3.1 Sosyal Ağ Analizi ve Merkezilik Ölçütleri	13
3.1.1 Derece Merkeziliği (Degree Centrality)	13
3.1.2 Yakınlık Merkeziliği (Closeness Centrality)	13
3.1.3 Arasındalık Merkeziliği (Betweenness Centrality)	14

3.1.4	Prestij Ölçütleri	14
3.2	PageRank Algoritması	15
3.2.1	Akış Formülasyonu (Flow Formulation)	15
3.2.2	Matris Formülasyonu	15
3.2.3	Güç İterasyonu (Power Iteration)	16
3.2.4	Rastgele Gezin Modeli ve Markov Zinciri	16
3.2.5	Örümcek Tuzakları (Spider Traps)	16
3.2.6	Çıkmaz Sokaklar (Dead Ends)	17
3.2.7	Google'ın Çözümü: Rastgele Atlama Modeli	17
3.2.8	Google Matrisi	17
3.2.9	Markov Zinciri Koşullarının Detaylı İncelenmesi	18
3.3	HITS Algoritması	18
3.3.1	Otoriteler ve Göbekler (Authorities and Hubs)	18
3.3.2	Algoritmanın Çalışması	19
3.3.3	Eş-atıf ve Bibliyografik Eşleşme ile İlişkisi	19
3.3.4	HITS'in Güçlü ve Zayıf Yönleri	20
3.4	Eş-atıf ve Bibliyografik Eşleşme Detayı	20
3.5	PageRank ve HITS'in Karşılaştırmalı Özeti	20
4	Web Crawler Mimarisi ve Bloom Filtreleri	22
4.1	Crawler Taksonomisi	22
4.2	Sıralı Crawler Mimarisi	22
4.3	URL İşleme	22
4.4	Metin İşleme	23
4.5	Örümcek Tuzakları	23
4.6	Sayfa Deposu	23
4.7	Eş Zamanlılık	23
4.8	Tazelik Politikaları	24
4.9	Gelişmiş Odaklanmış Tarama Algoritmaları	24
4.10	Bloom Filtreleri	24
4.10.1	Veri Yapısı ve Çalışma Prensibi	25
4.10.2	Olasılıksal Analiz: Dart Atma Modeli	25
4.10.3	Yanlış Pozitif Olasılığı	25
4.10.4	Sayısal Örnek	25
5	Web Kullanım Madenciliği	27
5.1	Giriş ve Temel Kavramlar	27
5.2	Üç Aşamalı Süreç	27
5.3	Veri Kaynakları	27
5.4	Kullanıcı Tanımlama Yöntemleri	27
5.5	Oturumlara Ayırma (Sessionization)	28
5.6	Önbellek Etkisi ve Yol Tamamlama (Path Completion)	29
5.7	Sayfa Görüntüleme (Pageview)	29
5.8	E-Ticaret Verisi ile Entegrasyon	29
5.9	Örüntü Keşfi için Madencilik Teknikleri	30
5.10	Özet	31
6	Tavsiye Sistemleri	32
6.1	Uzun Kuyruk (Long Tail) Fenomeni	32
6.2	Formal Model	32
6.3	Veri Toplama ve Seyreklik Problemi	32
6.4	İçerik Tabanlı Filtreleme (Content-Based Filtering)	32

6.4.1	Ana Fikir ve Madde Profilleri	32
6.4.2	Kullanıcı Profili ve Kosinüs Benzerliği ile Tahmin	33
6.4.3	Avantajlar ve Dezavantajlar	33
6.5	İşbirlikçi Filtreleme (Collaborative Filtering)	33
6.5.1	Kullanıcı-Kullanıcı (User-User) Yöntemi	33
6.5.2	Madde-Madde (Item-Item) Yöntemi	34
6.6	Değerlendirme Metrikleri	35
6.6.1	RMSE (Root Mean Square Error)	35
6.6.2	Precision at Top-K	35

1 Birliktelik Kuralları Madenciliği ve Apriori Algoritması

1.1 Pazar Sepeti Modeli ve Temel Kavramlar

Birliktelik kuralları madenciliği, Agrawal ve arkadaşları tarafından 1993 yılında önerilmiş olup veri madenciliğinin en kapsamlı çalışılan modellerinden biridir. Başlangıçta Pazar Sepeti Analizi (Market Basket Analysis) için geliştirilmiş olan bu yöntem, müşterilerin satın aldığı ürünler arasındaki ilişkileri keşfetmeyi amaçlar. Model, tüm verinin kategorik olduğunu varsayar ve sayısal veriler için iyi bir algoritma bulunmamaktadır.

Modelin temelinde bir ürün kümesi $I = \{i_1, i_2, \dots, i_m\}$ yer alır. Bir işlem (transaction) t , I 'nın bir alt kümesidir ($t \subseteq I$) ve işlem veritabanı $T = \{t_1, t_2, \dots, t_n\}$ işlemlerin bir kümesidir. Her işlem bir işlem kimliğine (TID) sahip olabilir. Örneğin bir süpermarket verisinde $t_1 = \{\text{ekmek, peynir, süt}\}$ bir işlemi, I ise mağazada satılan tüm ürünlerin kümesini temsil eder.

Bir ürün kümesi (itemset), I 'dan seçilen ürünlerin bir alt kümesidir. k adet ürün içeren bir ürün kümesine k -ürün kümesi (k -itemset) denir. Örneğin $\{\text{süt, ekmek, mısırgevreği}\}$ bir 3-ürün kümesidir. Bir X ürün kümesi, eğer $X \subseteq t$ ise t işlemi tarafından içeriliyor denir.

Birliktelik kuralı, $X \rightarrow Y$ biçiminde bir çıkarımdır; burada $X, Y \subset I$ ve $X \cap Y = \emptyset$ koşulu sağlanır. Yani bir kuralın sol ve sağ tarafındaki ürün kümeleri kesişmemelidir. Bir kuralın gücünü ölçmek için iki temel metrik kullanılır: destek (support) ve güven (confidence).

$$\text{destek}(X \cup Y) = \frac{|\{t \in T : X \cup Y \subseteq t\}|}{|T|} = P(X \cup Y) \quad (1)$$

Destek, $X \cup Y$ ürün kümesinin tüm işlemler içinde görülme olasılığıdır. Diğer bir deyişle, hem X hem de Y ürünlerini birlikte içeren işlemlerin oranıdır.

$$\text{güven}(X \rightarrow Y) = \frac{\text{destek}(X \cup Y)}{\text{destek}(X)} = P(Y | X) \quad (2)$$

Güven ise X ürünlerini içeren işlemler arasında Y ürünlerini de içerenlerin oranıdır; yani X verildiğinde Y 'nin koşullu olasılığıdır. Destek sayısı (support count) ise bir ürün kümesini içeren işlemlerin mutlak sayısıdır.

Birliktelik kuralları madenciliğinin amacı, kullanıcı tarafından belirlenen minimum destek (minsup) ve minimum güven (minconf) eşiklerini sağlayan tüm kuralları bulmaktır. Bu problemin iki temel özelliği tamlık (tüm kuralların bulunması) ve hedef ürün olmamasıdır — kuralın sağ tarafında herhangi bir ürün yer alabilir.

Örnek olarak minsup = %50 ve minconf = %50 olan bir veri kümesini ele alalım. Beş işlemten oluşan bir veritabanında A ürünü 3 kez, D ürünü 4 kez, AD birlikteliği ise 3 kez görülsün. Bu durumda $A \rightarrow D$ kuralı için destek $3/5 = \%60$, güven ise $3/3 = \%100$ olarak hesaplanır ve kural her iki eşiği de sağlar. Benzer şekilde $D \rightarrow A$ kuralı için destek yine %60 iken güven $3/4 = \%75$ olur.

1.2 Apriori Algoritması

Apriori algoritması, birliktelik kuralları madenciliğinde en bilinen ve en yaygın kullanılan algoritmadır. Algoritma iki ana adımdan oluşur: ilk adımda minimum destek eşiğini sağlayan tüm sık ürün kümeleri (frequent itemsets) bulunur; ikinci adımda ise bu sık ürün kümelerinden birliktelik kuralları türetilir. Sık ürün kümesi, desteği minsup değerine eşit veya daha büyük olan ürün kümesidir.

1.2.1 Apriori Özelliği ve Monotonluk

Apriori algoritmasının temelini oluşturan en önemli prensip aşağı kapama özelliği (downward closure property) olarak da adlandırılan apriori özelliğidir. Bu özellik şunu ifade eder: Eğer bir ürün kümesi sık ise, onun tüm alt kümeleri de sıktır. Bunun tersi olarak (contrapositive),

eğer bir ürün kümesi sık değilse, onu içeren hiçbir üst küme de sık olamaz. Bu monotonluk özelliği sayesinde aday ürün kümelerinin sayısı büyük ölçüde azaltılabilir. Örneğin $\{A, B\}$ ikilisi sık değilse, $\{A, B, C\}$ üçlüsü de sık olamaz çünkü bir üst kümenin desteği hiçbir zaman alt kümesinin desteğinden büyük olamaz.

1.2.2 Seviye-Seviye Arama ve Aday Üretimi

Algoritma seviye-seviye (level-wise) bir arama stratejisi izler; önce 1-ürün sık kümelerini, sonra 2-ürün sık kümelerini bularak ilerler. Her k iterasyonunda, bir önceki iterasyonda bulunan F_{k-1} sık kümelerinden yola çıkarak C_k aday kümeleri üretilir ve veritabanı taranarak bu adayların destekleri hesaplanır.

Aday üretim fonksiyonu (candidate-gen) iki alt adımdan oluşur: birleştirme (join) ve budama (prune). Birleştirme adımında, F_{k-1} içindeki yalnızca son elemanları farklı olan iki $(k-1)$ -ürün kümesi birleştirilerek k -ürün aday oluşturulur. Spesifik olarak, $f_1 = \{i_1, \dots, i_{k-2}, i_{k-1}\}$ ve $f_2 = \{i_1, \dots, i_{k-2}, i'_{k-1}\}$ şeklinde, ilk $k-2$ elemanı aynı ve $i_{k-1} < i'_{k-1}$ koşulunu sağlayan iki küme için $c = \{i_1, \dots, i_{k-1}, i'_{k-1}\}$ aday üretilir. Budama adımında ise, üretilen adayın tüm $(k-1)$ -elemanlı alt kümelerinin F_{k-1} içinde olup olmadığı kontrol edilir; eğer herhangi bir alt küme sık değilse, aday elenir.

Bu süreci somut bir örnekle açıklayalım. Varsayalım ki $F_3 = \{\{1, 2, 3\}, \{1, 2, 4\}, \{1, 3, 4\}, \{1, 3, 5\}, \{2, 3, 4\}\}$ olsun. Birleştirme adımında aynı ilk iki elemana sahip olan $\{1, 2, 3\}$ ile $\{1, 2, 4\}$ birleştirilerek $\{1, 2, 3, 4\}$ adayı, $\{1, 3, 4\}$ ile $\{1, 3, 5\}$ birleştirilerek ise $\{1, 3, 4, 5\}$ aday üretilir. Budama adımında $\{1, 3, 4, 5\}$ adayının $\{1, 4, 5\}$ alt kümesinin F_3 içinde olmadığı görülerek elenir. Sonuç olarak $C_4 = \{\{1, 2, 3, 4\}\}$ olur.

Algoritmanın sözde kodu aşağıdaki gibidir: C_1 tüm tekil ürünleri içerir ve ilk taramada F_1 bulunur. Ardından $k = 2$ 'den başlayarak $F_{k-1} \neq \emptyset$ olduğu sürece $C_k = \text{candidate-gen}(F_{k-1})$ üretilir, veritabanı taranarak adayların destek sayıları hesaplanır ve minsup eşliğini geçenler F_k olarak belirlenir. Algoritma en fazla K geçiş yapar; pratikte K genellikle 10 civarında sınırlıdır ve algoritma büyük veri kümelerine ölçeklenebilir.

Sayısal bir örnek üzerinde ilerleyelim. minsup = 0.5 olan dört işlemli bir veritabanı düşünelim: $T_{100} = \{1, 3, 4\}$, $T_{200} = \{2, 3, 5\}$, $T_{300} = \{1, 2, 3, 5\}$, $T_{400} = \{2, 5\}$. İlk taramada C_1 tekil ürün sayımları $\{1\} : 2$, $\{2\} : 3$, $\{3\} : 3$, $\{4\} : 1$, $\{5\} : 3$ olarak bulunur. minsup = 0.5 yani 2 işlem eşliğine göre $\{4\}$ elenir ve $F_1 = \{\{1\}, \{2\}, \{3\}, \{5\}\}$ olur. İkinci taramada F_1 'den üretilen $C_2 = \{\{1, 2\}, \{1, 3\}, \{1, 5\}, \{2, 3\}, \{2, 5\}, \{3, 5\}\}$ adaylarının sayımları sırasıyla 1, 2, 1, 2, 3, 2 bulunur ve minsup = 2 eşliğini geçenler $F_2 = \{\{1, 3\}, \{2, 3\}, \{2, 5\}, \{3, 5\}\}$ olarak belirlenir. Üçüncü taramada ise F_2 'den yalnızca $\{2, 3, 5\}$ aday üretilir (çünkü $\{1, 3\}$ ve $\{1, 5\}$ 'ten gelen $\{1, 3, 5\}$ adayının $\{1, 5\}$ alt kümesi sık değildir). $\{2, 3, 5\}$ üçlüsünün sayımı 2 çıkar ve $F_3 = \{\{2, 3, 5\}\}$ olur.

1.2.3 Sık Ürün Kümelerinden Kural Üretimi

Sık ürün kümeleri bulunduktan sonra, her bir sık X ürün kümesi için tüm boş olmayan öz alt kümeler $A \subset X$ dikkate alınır. $B = X - A$ olmak üzere, $A \rightarrow B$ kuralının güven değeri $\text{güven}(A \rightarrow B) = \text{destek}(X)/\text{destek}(A)$ formülü ile hesaplanır. Eğer bu değer minconf eşliğini geçerse kural kabul edilir. Bu adımda artık veritabanına tekrar bakılması gerekmez çünkü gerekli tüm destek bilgileri sık ürün kümesi madenciliği aşamasında kaydedilmiştir.

Örnek olarak $\{2, 3, 4\}$ ürün kümesinin %50 destekle sık olduğunu varsayalım. Alt kümelerin destekleri sırasıyla $\{2, 3\} : \%50$, $\{2, 4\} : \%50$, $\{3, 4\} : \%75$, $\{2\} : \%75$, $\{3\} : \%75$, $\{4\} : \%75$ olsun. Buradan türetilen kurallar ve güvenleri şöyledir: $2, 3 \rightarrow 4$ güveni $50/50 = \%100$, $2, 4 \rightarrow 3$ güveni $50/50 = \%100$, $3, 4 \rightarrow 2$ güveni $50/75 = \%67$, $2 \rightarrow 3, 4$ güveni $50/75 = \%67$, $3 \rightarrow 2, 4$ güveni $50/75 = \%67$, ve $4 \rightarrow 2, 3$ güveni $50/75 = \%67$. Tüm bu kuralların destek değeri %50'dir. minconf = %80 olsaydı yalnızca ilk iki kural kabul edilirdi.

1.3 Sayma Yöntemleri: Üçgensel Matris ve Üçlüler

Sık ürün kümelerini bulmanın en zor kısmı genellikle ikili (pair) sayımlarıdır çünkü ikililerin sayısı çok büyük olabilir. Örneğin 100.000 ürünlü bir sistemde yaklaşık 5 milyar olası ikili vardır. İkili sayımları için iki temel yaklaşım bulunmaktadır: üçgensel matris (triangular matrix) ve üçlüler tablosu (triples).

Üçgensel matris yönteminde tüm ürünler $1, 2, \dots, n$ şeklinde ardışık tam sayılarla numaralandırılır. Yalnızca $i < j$ olan $\{i, j\}$ çiftleri sayılır ve bu çiftler $\{1, 2\}, \{1, 3\}, \dots, \{1, n\}, \{2, 3\}, \dots, \{n-1, n\}$ sıralamasıyla bir dizide saklanır. $\{i, j\}$ çiftinin konumu $(i-1)(n-i/2) + j-i$ formülüyle hesaplanır. Toplam ikili sayısı $n(n-1)/2$ olduğundan, her ikili için 4 byte ayrılırsa yaklaşık $2n^2$ byte bellek gerekir. Bu yöntem tüm olası çiftleri sayar ve bellek kullanımı ürün sayısının karesiyle orantılıdır.

Üçlüler tablosu yönteminde ise yalnızca gerçekten görülen (sayımı sıfırdan büyük olan) çiftler için $[i, j, c]$ üçlülere saklanır; burada c sayım değeridir. Her üçlü yaklaşık 12 byte yer kaplar (üç adet 4 byte'lık tam sayı). Toplam bellek kullanımı yaklaşık $12p$ olup burada p gerçekten görülen ikili sayısıdır. Eğer olası çiftlerin en fazla $1/3$ 'ü gerçekten görülüyorsa üçlüler yöntemi üçgensel matris yönteminden daha az bellek kullanır. Ancak üçlüler yöntemi verimli erişim için bir hash tablosu gibi ek yapılar gerektirebilir.

Apriori algoritması ile ikili sayımında, yalnızca her iki tekil ürünü de sık olan çiftler sayılır. Bu sayede sayılacak çiftlerin sayısı büyük ölçüde azalır. Sık ürünler yeniden $1, 2, \dots, m$ şeklinde numaralandırılırsa (m sık ürün sayısıdır), üçgensel matris yöntemi yaklaşık $2m^2$ byte bellek kullanır ki bu genellikle ana belleğe sığabilir.

1.4 PCY Algoritması

PCY (Park-Chen-Yu) algoritması, Apriori'nin ilk geçişinde ana belleğin büyük kısmının atıl durumda olmasından faydalanarak geliştirilmiş bir iyileştirmedir. Algoritmanın temel fikri, ilk geçişte yalnızca tekil ürünleri değil, aynı zamanda ikili çiftlerinin hash değerlerini de saymaktır.

PCY algoritmasının birinci geçişinde, tüm sepetler okunurken her bir ürünün sayımı yapılır ve eş zamanlı olarak sepetteki tüm ikililer bir hash fonksiyonundan geçirilerek ilgili hash kovalarının sayacı bir artırılır. Burada yalnızca kova sayımları tutulur; kovalara düşen ikililerin kendileri saklanmaz. Bir kovanın sayımı destek eşğine eşit veya büyükse o kova sık kova (frequent bucket) olarak adlandırılır. Bir kova sık değilse, o kovaya hashlenen hiçbir ikili sık olamaz — bu, apriori özelliğinin hash kovalarına uygulanmış halidir.

Birinci geçişin sonunda, kova sayımları 4 byte'lık tam sayılar yerine bir bit vektörüne (bitmap) dönüştürülür. Sık kovalar için 1, sık olmayanlar için 0 değeri atanır. Bu sayede bellek kullanımı $1/32$ oranına düşer. Ayrıca hangi ürünlerin sık olduğu belirlenir ve ikinci geçiş için listelenir.

İkinci geçişte yalnızca şu iki koşulu sağlayan ikililer sayılır: (1) ikilinin her iki ürünü de sık olmalıdır, (2) ikilinin hash değerine karşılık gelen bitmap biti 1 olmalıdır. Bu iki filtre sayesinde sayılması gereken aday ikili sayısı önemli ölçüde azalır. PCY'nin Apriori'den daha iyi performans göstermesi için hash tablosunun aday ikililerin en az $2/3$ 'ünü elemesi gerekir. Bunun nedeni, PCY'nin ikinci geçişte üçlüler tablosu kullanmak zorunda olmasıdır (üçgensel matris yöntemi hash filtrelemesiyle uyumlu değildir).

PCY'nin ilk geçişinde bellek tahsisi şu şekildedir: ürün sayımları için ürün başına 4 byte ayrılır, kalan tüm bellek ise hash kovaları için kullanılır. Kova sayısı ne kadar fazla olursa, her bir kovaya düşen ortalama ikili sayısı o kadar azalır ve sık olmayan kovaların eleddiği ikili sayısı artar. Öte yandan kova sayımlarının destek eşğini aşması gerekmez; sayım değeri s değerine ulaştığında sayacın daha fazla artırılmasına gerek yoktur.

1.5 Multistage Algoritması

Multistage (Çok Aşamalı) algoritması, PCY'nin üç geçişli bir uzantısıdır. Temel fikir şudur: PCY'nin birinci geçişinden sonra yalnızca PCY'nin ikinci geçişinde sayılacak aday ikilileri kul-

lanarak ikinci bir hash tablosu oluşturulur. Bu ara geçişte daha az ikili işlendiği için hash kovalarındaki yanlış pozitifler (sık kova görünüp içinde sık ikili olmaması durumu) azalır.

Multistage algoritması üç geçişten oluşur. Birinci geçiş tıpkı PCY'nin birinci geçişi gibidir: ürün sayımları ve birinci hash tablosu oluşturulur, birinci bitmap üretilir. İkinci geçişte yalnızca birinci bitmap'ten geçen ikililer ikinci bir hash fonksiyonuyla hashlenir ve ikinci bir hash tablosu oluşturulur; bu geçişte ikililerin destek sayımları yapılmaz, yalnızca ikinci hash tablosu doldurulur ve ikinci bitmap üretilir. Üçüncü geçişte ise her iki bitmap'te de 1 olan kovalara düşen ve her iki ürünü de sık olan ikililerin gerçek sayımları yapılır.

Multistage'de iki hash fonksiyonunun bağımsız olması kritik önem taşır. Aksi takdirde birinci hash'e göre sık olmayan bir kovaya düşen ikili, ikinci hash'te sık bir kovaya düşerse yanlışlıkla sayıma dahil edilebilir. Bu nedenle üçüncü geçişte her ikili için her iki hash değeri de kontrol edilmelidir.

1.6 Multihash Algoritması

Multihash (Çoklu Hash) algoritması, Multistage'in üç geçiş yerine iki geçişte benzer bir fayda sağlamayı hedefleyen bir varyantıdır. Temel fikir, birinci geçişte ana belleği iki (veya daha fazla) bağımsız hash tablosu arasında bölüştürmektir.

Birinci geçişte, ürün sayımları için ayrılan belleğin kalan kısmı iki bağımsız hash tablosu için kullanılır. Her sepetteki tüm ikililer her iki hash fonksiyonuyla da hashlenir ve ilgili kovaların sayaçları artırılır. Geçiş sonunda her iki hash tablosu için de ayrı ayrı bitmap üretilir. İkinci geçişte ise her iki bitmap'te de 1 olan kovalara düşen ve her iki ürünü de sık olan ikililer sayılır.

Multihash'in riski, hash tablolarının sayısını artırmanın her bir tablodaki kova sayısını azaltmasıdır. Kova sayısının azalması, kova başına düşen ortalama ikili sayısını artırır ve daha fazla kovanın destek eşiğini aşmasına (dolayısıyla daha fazla yanlış pozitif) neden olabilir. Bu nedenle Multihash yalnızca kovaların büyük çoğunluğu hâlâ destek eşliğinin altında kalabildiğinde etkilidir. Avantajı ise Multistage'in üç geçişine kıyasla yalnızca iki geçiş gerektirmesidir; disk tabanlı sistemlerde geçiş sayısı maliyetin ana belirleyicisi olduğundan bu önemli bir kazançtır.

1.7 Sıralı Örüntü Madenciliği (Sequential Pattern Mining)

Birliktelik kuralları madenciliği işlemlerin sırasını dikkate almaz. Oysa birçok uygulamada işlemlerin zamansal sıralaması kritik öneme sahiptir. Örneğin pazar sepeti analizinde müşterilerin önce yatak, bir süre sonra ise çarşaf alması gibi sıralı satın alma davranışları, ya da Web kullanım madenciliğinde kullanıcıların bir Web sitesindeki sayfa ziyaret sıraları önemli örüntüler barındırabilir.

Sıralı örüntü madenciliğinde temel kavramlar şöyle tanımlanır: $I = \{i_1, i_2, \dots, i_m\}$ ürün kümesi olmak üzere, bir eleman (element/itemset) I 'nin boş olmayan bir alt kümesidir ve $\{x_1, x_2, \dots, x_k\}$ şeklinde gösterilir. Bir dizi (sequence) s , elemanların sıralı bir listesidir ve $\langle a_1 a_2 \dots a_r \rangle$ ile gösterilir; burada her a_i bir elemandır. Bir dizinin boyutu (size) içerdiği eleman sayısı, uzunluğu (length) ise içerdiği toplam ürün sayısıdır. Uzunluğu k olan diziyi k -dizisi denir.

$s_1 = \langle a_1 a_2 \dots a_r \rangle$ dizisi, eğer $1 \leq j_1 < j_2 < \dots < j_r \leq v$ olacak şekilde her i için $a_i \subseteq b_{j_i}$ koşulunu sağlayan indisler varsa, $s_2 = \langle b_1 b_2 \dots b_v \rangle$ dizisinin bir alt dizisidir (subsequence). Bu durumda s_2 , s_1 'i içeriyor denir.

Örnek vermek gerekirse, $I = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ olsun. $\langle \{3\}\{4, 5\}\{8\} \rangle$ dizisi, $\langle \{6\}\{3, 7\}\{9\}\{4, 5, 8\}\{3, 8\} \rangle$ dizisinin bir alt dizisidir çünkü $\{3\} \subseteq \{3, 7\}$, $\{4, 5\} \subseteq \{4, 5, 8\}$ ve $\{8\} \subseteq \{3, 8\}$ koşulları sağlanır ve indisler artan sıradadır. Bu dizinin boyutu 3, uzunluğu ise 4'tür (1+2+1).

Sıralı örüntü madenciliğinin amacı, bir dizi veritabanında kullanıcı tarafından belirlenen minimum destek eşliğini sağlayan tüm sık dizileri (sequential patterns) bulmaktır. Bir dizinin desteği, veritabanındaki toplam dizilerden onu alt dizi olarak içerenlerin oranıdır.

GSP (Generalized Sequential Pattern) algoritması, sıralı örüntü madenciliği için Apriori benzeri bir yaklaşımdır. Apriori’de olduğu gibi seviye-seviye arama yapar: önce 1-elemanlı sık diziler bulunur, ardından k -elemanlı sık dizilerden $(k + 1)$ -elemanlı adaylar üretilir ve veritabanı taranarak destekleri hesaplanır. GSP’nin Apriori’den temel farkı, aday üretiminde dizilerin sıralı yapısını dikkate almasıdır; aday üretimi için iki temel işlem kullanılır: iki dizinin birleştirilmesiyle eleman sayısı artırılabilir ya da bir diziye yeni bir eleman eklenebilir.

1.8 Sınıf Birliktelik Kuralları (CAR)

Normal birliktelik kuralları madenciliğinde bir hedef değişken yoktur; herhangi bir ürün kuralın hem koşul hem de sonuç kısmında yer alabilir. Ancak bazı uygulamalarda kullanıcı belirli hedef sınıflarla ilgilenir. Örneğin, bilinen bazı konu başlıklarına ait metin belgelerinde hangi kelimelerin hangi konularla ilişkili olduğu merak edilebilir.

Sınıf birliktelik kuralları (Class Association Rules — CAR) modelinde, her işlem bir $y \in Y$ sınıf etiketi ile etiketlenmiştir; burada $I \cap Y = \emptyset$ koşulu sağlanır. Bir sınıf birliktelik kuralı $X \rightarrow y$ biçimindedir; $X \subseteq I$ ürün kümesi, $y \in Y$ ise bir sınıf etiketidir. Destek ve güven tanımları normal birliktelik kurallarıyla aynıdır.

Örnek olarak 7 belgeden oluşan bir metin veri kümesini ele alalım. doc1: {Student, Teach, School}:Education, doc2: {Student, School}:Education, doc3: {Teach, School, City, Game}:Education, doc4: {Baseball, Basketball}:Sport, doc5: {Basketball, Player, Spectator}:Sport, doc6: {Baseball, Coach, Game, Team}:Sport, doc7: {Basketball, Team, City, Game}:Sport. minsup = %20 (yani en az 2 işlem) ve minconf = %60 için örnek CAR kuralları şunlardır: Student, School $\rightarrow$ Education [destek = 2/7, güven = 2/2 = %100] ve Game $\rightarrow$ Sport [destek = 3/7, güven = 2/3 $\approx$ %67].

CAR madenciliği normal birliktelik kurallarının aksine tek adımda gerçekleştirilebilir. Temel işlem, desteği minsup üzerinde olan tüm kural öğelerini (ruleitems) bulmaktır. Bir kural öğesi (condset, y) biçimindedir; burada condset $\subseteq I$ bir koşul kümesi ve $y \in Y$ bir sınıf etiketidir. Her kural öğesi aslında condset $\rightarrow y$ kuralını temsil eder. Apriori algoritması, sık kural öğelerini bulmak için uyarlanabilir; aday üretimi ve budama adımları benzer şekilde çalışır ancak sınıf etiketi her zaman kural öğesinin bir parçası olarak taşınır.

1.9 Veri Formatı Dönüşümleri

Birliktelik kuralları madenciliğinde veriler işlem formunda (transaction form) ya da tablo formunda (table form) olabilir. İşlem formunda her satır bir işlemi temsil eder ve o işlemde yer alan ürünler listelenir; örneğin a, b veya a, c, d, e gibi. Tablo formunda ise her satır bir kaydı, her sütun bir niteliği (attribute) temsil eder ve hücrelerde nitelik değerleri yer alır.

Birliktelik kuralları madenciliği için tablo formundaki verilerin işlem formuna dönüştürülmesi gerekir. Bu dönüşüm her bir tablo satırı için (nitelik, değer) çiftlerinin oluşturulmasıyla yapılır. Örneğin Attr1, Attr2, Attr3 sütunlarına sahip ve değerleri (a, b, d) ile (b, c, e) olan iki satırlı bir tablo, işlem formunda $(Attr1, a), (Attr2, b), (Attr3, d)$ ve $(Attr1, b), (Attr2, c), (Attr3, e)$ şeklinde iki işleme dönüştürülür. Bu dönüşüm sayesinde nitelik-değer çiftleri birer ürün gibi ele alınabilir ve standart birliktelik kuralları algoritmaları uygulanabilir.

2 Bilgi Erişimi ve Metin İşleme

2.1 Giriş ve DIKW Hiyerarşisi

Bilgi erişimi (Information Retrieval – IR), kullanıcıların bilgi ihtiyaçlarını karşılayan dokümanları bulma bilimidir. Kullanıcı ihtiyacı bir sorgu (query) olarak ifade edilir ve sistem bu sorguyla en ilgili dokümanları döndürür. Tarihsel olarak IR, dokümanı temel birim kabul eden doküman erişimi üzerine kurulmuştur; teknik olarak ise bilginin elde edilmesi, düzenlenmesi, saklanması, erişilmesi ve dağıtılması süreçlerini inceler. Metin madenciliği ve web araması da köklerini IR'den alır.

Bilgi erişimi sürecini anlamak için DIKW hiyerarşisi kullanışlı bir çerçeve sunar. Bu hiyerarşi dört katmandan oluşur: Veri (Data), ham web sayfalarından ve işlenmemiş metinlerden oluşur. Enformasyon (Information), ham veriye bir sorgu uygulandığında elde edilen sonuçtur; örneğin bir arama motoruna "veri madenciliği" yazdığınızda dönen sayfalar enformasyondur. Bilgi (Knowledge), kullanıcının bu enformasyonu kendi bağlamında işleyip anlamlandırmasıyla oluşur. Bilgelik (Wisdom) ise birçok kullanıcının benzer eylemlerinin sentezinden doğan, uzun vadeli ve derinlemesine kavrayıştır. IR sistemleri özellikle veriden enformasyona geçiş katmanında çalışır.

2.2 Bilgi Erişimi ve Veritabanı Erişimi Karşılaştırması

IR ile geleneksel veritabanı erişimi arasında temel farklar vardır. Veritabanı sistemleri yapılandırılmış (structured) veri üzerinde çalışır; SQL gibi kesin sorgu dilleri kullanılır ve sonuçlar tam eşleşmeye (exact match) dayanır. Bir kayıt ya sorguyu tam olarak karşılar ya da karşılamaz. IR sistemleri ise yapılandırılmamış (unstructured) veya yarı yapılandırılmış metin üzerinde çalışır. Sorgular genellikle anahtar kelime tabanlıdır ve sonuçlar kısmi eşleşmeye (partial match) dayanır; dokümanlar ilgililik derecesine göre sıralanır (ranking). Veritabanında arama kesin ve deterministiktir, IR'de ise sonuçlar belirsizlik içerir ve en iyi eşleşme ilkesine göre çalışır. Ayrıca IR sistemlerinde sorguların kendisi de genellikle eksik veya belirsiz olabilir; kullanıcı ne aradığını tam olarak ifade edemeyebilir. Bu nedenle IR modelleri doküman-sorgu benzerliğini sayısal bir skorla ifade eder.

2.3 Boolean Modeli

Boolean modeli, en basit ve en eski IR modellerinden biridir. Bu modelde her doküman ve sorgu, bir "kelime torbası" (bag of words) olarak ele alınır; kelime sıralaması dikkate alınmaz. D doküman koleksiyonu için $V = \{t_1, t_2, \dots, t_{|V|}\}$ kümesi, koleksiyondaki tüm benzersiz terimlerin (kelimelerin) sözlüğüdür (vocabulary). Her $d_j \in D$ dokümanı, bir ağırlık vektörü ile temsil edilir: $d_j = (w_{1j}, w_{2j}, \dots, w_{|V|j})$. Boolean modelinde $w_{ij} \in \{0, 1\}$ değerini alır; terim t_i dokümanda d_j geçiyorsa $w_{ij} = 1$, geçmiyorsa $w_{ij} = 0$ olur.

Sorgular AND, OR ve NOT Boolean operatörleriyle oluşturulur. Örneğin ((data AND mining) AND (NOT text)) sorgusu, "data" ve "mining" terimlerini içeren ama "text" terimini içermeyen dokümanları getirir. Sistem, sorguyu mantıksal olarak doğru (true) yapan her dokümanı döndürür; buna tam eşleşme (exact match) denir. Boolean modelinin en büyük avantajı basitliği ve hesaplama verimliliğidir. Ancak iki önemli dezavantajı vardır: terim frekansı dikkate alınmadığı için bir terimin dokümanda kaç kez geçtiği önemsizdir ve sonuçlarda sıralama yapılamaz. Bir doküman ya eşleşir ya eşleşmez; kısmi ilgililik derecesi yoktur. Örneğin "hardware" kelimesi bir dokümanda 1 kez, başka bir dokümanda 50 kez geçse de Boolean modelinde ikisi de aynı ağırlığa sahiptir. Bu nedenle Boolean modeli tek başına büyük koleksiyonlarda yetersiz kalır; ancak diğer modellerle hibrit olarak halen kullanılmaktadır.

2.4 Vektör Uzay Modeli ve TF-IDF

Vektör uzay modeli (Vector Space Model – VSM), Boolean modelinin en büyük eksikliğini giderir: terim ağırlıkları artık yalnızca 0 veya 1 değil, sürekli değerler alır. Bu sayede hem kısmi eşleşme mümkün olur hem de dokümanlar ilgililik derecesine göre sıralanabilir.

2.4.1 TF (Terim Frekansı) Ağırlıklandırması

En basit ağırlıklandırma şeması Ham TF'dir: bir t_i teriminin d_j dokümanındaki ağırlığı, o terimin dokümanda görülme sayısına eşittir; $w_{ij} = f_{ij}$ ile gösterilir. Ancak ham TF, uzun dokümanlara haksız avantaj sağlar; bu nedenle genellikle normalizasyon uygulanır. En yaygın normalizasyon yöntemi, TF değerini dokümandaki toplam terim sayısına bölmektir. Ayrıca log-normalizasyon da kullanılır: $1 + \log(f_{ij})$ (eğer $f_{ij} > 0$ ise) veya 0 (eğer $f_{ij} = 0$ ise). Bu dönüşüm, çok yüksek frekanslı terimlerin etkisini bastırır. Örneğin bir terim bir dokümanda 100 kez geçiyorsa log-normalize TF değeri $1 + \log(100) \approx 5.6$ olurken, 1 kez geçiyorsa $1 + \log(1) = 1$ olur; yani 100 kat daha fazla geçmesine rağmen ağırlığı yalnızca 5.6 kat fazladır.

2.4.2 IDF (Ters Doküman Frekansı)

TF tek başına yeterli değildir; çünkü "the", "ve", "bir" gibi çok sık geçen kelimeler hemen her dokümanda yüksek TF değerine sahip olur ama ayırt edici değildir. IDF (Inverse Document Frequency) bu sorunu çözer. Bir t_i teriminin IDF değeri şöyle hesaplanır:

$$\text{IDF}(t_i) = \log \left(\frac{N}{\text{df}_i} \right) \quad (3)$$

Burada N toplam doküman sayısı, df_i ise t_i terimini içeren doküman sayısıdır. Eğer bir terim tüm dokümanlarda geçiyorsa ($\text{df}_i = N$), IDF değeri $\log(1) = 0$ olur ve terimin ağırlığa katkısı sıfırlanır. Nadir terimler ise yüksek IDF değerine sahip olur. Örneğin 1000 dokümanlık bir koleksiyonda "algoritma" kelimesi 10 dokümanda geçiyorsa $\text{IDF} = \log(1000/10) = \log(100) = 2$, "veri" kelimesi 500 dokümanda geçiyorsa $\text{IDF} = \log(1000/500) = \log(2) \approx 0.301$ olur. Görüldüğü gibi "algoritma" daha ayırt edici bir terimdir ve çok daha yüksek IDF ağırlığı alır.

2.4.3 TF-IDF Birleşik Ağırlığı

Bir terimin nihai TF-IDF ağırlığı, TF ve IDF değerlerinin çarpımıdır:

$$w_{ij} = \text{TF}(t_i, d_j) \times \text{IDF}(t_i) = f_{ij} \cdot \log \left(\frac{N}{\text{df}_i} \right) \quad (4)$$

Bu ağırlıklandırma şemasının arkasındaki sezgi şudur: bir terim, içinde bulunduğu dokümanda sık geçiyorsa (yüksek TF) ve koleksiyon genelinde nadir ise (yüksek IDF), o terim o doküman için çok önemlidir. TF-IDF, modern IR sistemlerinin temel taşıdır ve web arama motorlarından metin sınıflandırmaya kadar pek çok alanda kullanılır.

2.4.4 Kosinüs Benzerliği ile Erişim

Vektör uzay modelinde hem dokümanlar hem de sorgu aynı $|V|$ boyutlu uzayda birer vektör olarak temsil edilir. Doküman $d_j = (w_{1j}, w_{2j}, \dots, w_{|V|j})$ ve sorgu $q = (w_{1q}, w_{2q}, \dots, w_{|V|q})$ şeklindedir. Bir dokümanın sorguyla ilgililiği, iki vektör arasındaki kosinüs benzerliği ile ölçülür:

$$\cos(q, d_j) = \frac{q \cdot d_j}{\|q\| \cdot \|d_j\|} = \frac{\sum_{i=1}^{|V|} w_{iq} \cdot w_{ij}}{\sqrt{\sum_{i=1}^{|V|} w_{iq}^2} \cdot \sqrt{\sum_{i=1}^{|V|} w_{ij}^2}} \quad (5)$$

Kosinüs benzerliği $[-1, 1]$ aralığında değer alır, ancak TF-IDF ağırlıkları negatif olmadığı için pratikte $[0, 1]$ aralığındadır. Değer 1'e yaklaştıkça doküman sorguya daha benzerdir. Vektör uzunluğuna bölme işlemi, uzun dokümanların yalnızca daha fazla terim içerdikleri için yüksek benzerlik skoru almasını engeller.

Somut bir örnek verelim: Sözlüğümüz $V = \{\text{hardware, software, users}\}$ olsun. Doküman vektörlerimiz $A1=(1,0,0)$, $A2=(0,1,0)$, $A3=(0,0,1)$, $A4=(1,1,0)$, $A5=(1,0,1)$, $A6=(0,1,1)$, $A7=(1,1,1)$ şeklinde tanımlansın. Sorgumuz "hardware AND software" yani $q = (1, 1, 0)$ olsun. Boolean modelinde "AND" ile yalnızca $A4$ ve $A7$ dönerken, "OR" ile $A3$ hariç tüm dokümanlar döner ve aralarında sıralama yapılamaz. Kosinüs benzerliği ile hesaplırsak: $\cos(q, A1) = (1 \cdot 1 + 1 \cdot 0 + 0 \cdot 0) / (\sqrt{2} \cdot \sqrt{1}) = 1/\sqrt{2} \approx 0.71$, $\cos(q, A4) = (1 + 1 + 0) / (\sqrt{2} \cdot \sqrt{2}) = 2/2 = 1.0$, $\cos(q, A7) = (1 + 1 + 0) / (\sqrt{2} \cdot \sqrt{3}) = 2/\sqrt{6} \approx 0.82$. Sıralı sonuç kümesi $\{A4(1.0), A7(0.82), A1(0.71), A2(0.71), A5(0.5), A6(0.5)\}$ olur; $A3$ 'ün benzerliği sıfırdır. Vektör uzay modeli böylece hem sıralama yapabilir hem de "AND" ile elenecek $A1$, $A2$ gibi kısmen ilgili dokümanları makul bir skorla listeleyebilir.

2.5 Okapi BM25

Okapi BM25, TF-IDF ailesinin en başarılı ve yaygın kullanılan varyantlarından biridir. BM25, doğrudan bir ilgililik skoru hesaplar ve iki önemli iyileştirme getirir: TF doygunluğu (saturation) ve doküman uzunluk normalizasyonu. Bir q sorgusu için bir d dokümanın BM25 skoru şöyle hesaplanır:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdI}}\right)} \quad (6)$$

Burada $f(t, d)$ terimin dokümandaki frekansı, $|d|$ doküman uzunluğu, avgdI ortalama doküman uzunluğu, k_1 ve b ise ayarlanabilir parametrelerdir (tipik değerler $k_1 \in [1.2, 2.0]$ ve $b = 0.75$). IDF bileşeni BM25'te genellikle şu şekilde hesaplanır:

$$\text{IDF}(t) = \log \left(\frac{N - \text{df}_t + 0.5}{\text{df}_t + 0.5} \right) \quad (7)$$

k_1 parametresi TF doygunluğunu kontrol eder: bir terimin dokümanda çok fazla geçmesinin marjinal faydasını sınırlar. b parametresi ise doküman uzunluk normalizasyonunun derecesini belirler; $b = 1$ tam normalizasyon, $b = 0$ ise normalizasyonsuz durumdur. BM25'in temel felsefesi, bir terimin ilk birkaç geçişinin çok değerli olduğu, ancak sonraki geçişlerin giderek azalan katkı sağladığıdır. Bu özellik, TF-IDF'nin doğrusal yapısına göre daha gerçekçi bir modelleme sunar.

2.6 Rocchio İlgililik Geri Bildirimi

İlgililik geri bildirimi (relevance feedback), erişim etkinliğini artırmak için kullanılan bir tekniktir. Süreç üç adımda işler: (1) Kullanıcı, ilk erişim sonuçları arasından ilgili bulduğu dokümanları (D_r) ve ilgisiz bulduğu dokümanları (D_{ir}) işaretler. (2) Sistem, bu geri bildirimi kullanarak orijinal sorgu vektörünü genişletir. (3) Genişletilmiş sorgu ile ikinci bir erişim turu yapılır.

Rocchio yöntemi, sorgu genişletme için klasik bir formüldür. α , β ve γ parametreleriyle kontrol edilen Rocchio formülü şudur:

$$q_m = \alpha \cdot q_0 + \beta \cdot \frac{1}{|D_r|} \sum_{d_j \in D_r} d_j - \gamma \cdot \frac{1}{|D_{ir}|} \sum_{d_j \in D_{ir}} d_j \quad (8)$$

Burada q_0 orijinal sorgu vektörü, q_m değiştirilmiş (genişletilmiş) sorgu vektörüdür. α parametresi orijinal sorgunun ağırlığını, β ilgili dokümanlardan gelen pozitif katkıyı, γ ise ilgisiz dokümanlardan gelen negatif katkıyı kontrol eder. Tipik olarak $\alpha = 1$, $\beta = 0.75$, $\gamma = 0.15$ gibi değerler

kullanılır; yani orijinal sorgu korunur, ilgili dokümanlar sorguyu güçlendirir, ilgisiz dokümanlar ise sorgudan uzaklaştırır. Bu yöntem sayesinde, kullanıcının ilk sorguda ifade edemediği ama ilgili dokümanlarda bulunan terimler otomatik olarak sorguya eklenir ve ikinci turda daha iyi sonuçlar elde edilir.

2.7 Metin Ön İşleme

IR ve metin madenciliğinde ham metnin doğrudan kullanılması verimsiz ve etkisizdir. Bu nedenle bir dizi ön işleme adımı uygulanır: kelime çıkarma (tokenization), durma kelimelerinin çıkarılması (stopwords removal), gövdeleme (stemming) ve frekans sayımı ile TF-IDF ağırlıklarının hesaplanması.

2.7.1 Durma Kelimelerinin Çıkarılması

Durma kelimeleri (stopwords), İngilizcede "the", "of", "and", "to", "a", "in" gibi çok sık kullanılan ve bilgi taşımayan kelimelerdir. Tipik olarak 400–500 kelimelik bir durma listesi kullanılır; uygulamaya özel ek durma kelimeleri de tanımlanabilir. Durma kelimelerinin çıkarılmasının iki temel nedeni vardır: (1) İndeks boyutunu küçültmek – durma kelimeleri toplam kelime sayısının yaklaşık %20–30'unu oluşturur. (2) Etkinliği ve verimliliği artırmak – durma kelimeleri arama ve metin madenciliği için yararlı değildir, hatta erişim sistemini yavaşlatırlar. Örneğin "the" kelimesi hemen her dokümanda geçtiği için IDF değeri sıfıra yakındır ve TF-IDF ağırlığı anlamsızlaşır; bu tür kelimeleri indeks dışında tutmak hem depolama hem hesaplama açısından tasarruf sağlar.

2.7.2 Gövdeleme

Gövdeleme (stemming), bir kelimenin kökünü veya gövdesini bulma tekniğidir. Örneğin "user", "users", "used", "using", "usability" kelimelerinin ortak gövdesi "use"tur; "engineering", "engineered", "engineer" kelimelerinin gövdesi ise "engineer"dir. Gövdelemenin iki temel faydası vardır: (1) Benzer anlamlı kelimeleri eşleştirerek erişim etkinliğini, özellikle de geri çağırma (recall) oranını artırır. (2) Aynı köke sahip kelimeleri birleştirerek indeks boyutunu %40–50'ye varan oranlarda küçültür.

Temel gövdeleme yöntemleri kural tabanlıdır. Örneğin: bir kelime "s" dışında bir sessiz harfle bitiyor ve ardından "s" geliyorsa "s"i sil; kelime "es" ile bitiyorsa "s"i düşür; kelime "ing" ile bitiyorsa ve kalan kısım tek bir harf veya "th" değilse "ing"i sil; kelime bir sessiz harften sonra "ed" ile bitiyorsa ve bu işlem tek harf bırakmıyorsa "ed"i sil. Ayrıca dönüşüm kuralları da vardır: örneğin "ies" ile biten ama "eies" veya "aies" ile bitmeyen kelimelerde "ies" → "y" dönüşümü yapılır.

İki yaygın gövdeleme algoritması şunlardır: Porter Stemmer, 1980 yılında Martin Porter tarafından geliştirilmiş ve İngilizce için en çok kullanılan gövdeleme algoritmasıdır. Beş aşamalı bir kural zinciri uygular; her aşamada belirli son ekleri (suffix) kaldırır veya dönüştürür. Örneğin "-ational" → "-ate", "-ization" → "-ize", "-fulness" → "-ful" gibi dönüşümler yapar. Porter algoritması oldukça agresiftir ve bazen aşırı gövdelemeye neden olabilir. Snowball (Porter2) ise Porter'ın geliştirilmiş halidir ve çok dilli destek sunar. Snowball çerçevesinde İngilizce, Fransızca, İspanyolca, Almanca, Türkçe gibi pek çok dil için gövdeleme algoritmaları tanımlanmıştır. Snowball'da kurallar daha düzenli bir yapıda ifade edilir ve her dilin morfolojik özelliklerine göre uyarlanabilir.

2.8 Ters İndeks Yapısı

Ters indeks (inverted index), büyük ölçekli IR sistemlerinin temel veri yapısıdır. Temel fikir şudur: her benzersiz terim, o terimi içeren tüm dokümanların bir listesine bağlanır. Bu, standart

bir indeksin (dokümandan terimlere) tam tersidir. Bir $D = \{d_1, d_2, \dots, d_N\}$ doküman koleksiyonu için ters indeks yapısı şöyledir:

Her bir $t_i \in V$ terimi için bir gönderi listesi (posting list) tutulur. Bu liste, t_i 'yi içeren dokümanların kimliklerini (ve genellikle terimin o dokümandaki frekansını ve pozisyonlarını) içerir. Örneğin "algorithm" terimi için gönderi listesi $\{(d_3, 5), (d_7, 2), (d_{12}, 8)\}$ şeklinde olabilir; bu, "algorithm" teriminin d_3 'te 5 kez, d_7 'de 2 kez ve d_{12} 'de 8 kez geçtiğini gösterir.

Ters indeksin en büyük avantajı, bir sorgu terimini içeren dokümanları sabit zamanda (constant time) bulabilmesidir. Sorgu işleme üç adımda gerçekleşir: (1) Sözlük araması: sorgudaki her terim için ters indeksten ilgili gönderi listesi alınır. (2) Sonuç birleştirme: birden fazla sorgu terimi varsa, gönderi listeleri kesiştirilir (AND) veya birleştirilir (OR). (3) Sıralama skoru hesaplama: elde edilen dokümanlar için içerik tabanlı (TF-IDF, BM25 gibi) veya bağlantı tabanlı (PageRank gibi) sıralama skorları hesaplanır ve sonuçlar sıralanır.

2.9 Değerlendirme: Kesinlik ve Geri Çağırma

IR sistemlerinin performansını ölçmek için en temel iki metrik kesinlik (precision) ve geri çağırma (recall). Bir sorgu için ilgili dokümanların tam kümesi bilindiğinde bu metrikler şöyle tanımlanır. R ilgili dokümanlar kümesi, S ise sistemin döndürdüğü dokümanlar kümesi olsun.

$$\text{Kesinlik (Precision)} = \frac{|R \cap S|}{|S|} = \frac{\text{getirilen ilgili doküman sayısı}}{\text{getirilen toplam doküman sayısı}} \quad (9)$$

$$\text{Geri Çağırma (Recall)} = \frac{|R \cap S|}{|R|} = \frac{\text{getirilen ilgili doküman sayısı}}{\text{toplam ilgili doküman sayısı}} \quad (10)$$

Kesinlik, "getirilen dokümanların ne kadarı gerçekten ilgili?" sorusuna yanıt verir ve sistemin isabet oranını ölçer. Geri çağırma ise "ilgili dokümanların ne kadarı getirildi?" sorusuna yanıt verir ve sistemin kapsamını ölçer. Bu iki metrik genellikle ters orantılıdır: kesinliği artırmak için daha seçici davranıldığında geri çağırma düşer, geri çağırma artırmak için daha fazla doküman getirildiğinde ise kesinlik düşer.

Somut bir örnek: bir koleksiyonda bir sorguyla ilgili toplam 20 doküman olsun ($|R| = 20$). Sistem 15 doküman getirsin ($|S| = 15$) ve bunların 12'si gerçekten ilgili olsun ($|R \cap S| = 12$). Bu durumda Kesinlik $= 12/15 = 0.80$ (%80), Geri Çağırma $= 12/20 = 0.60$ (%60) olur. Sistem ilgili dokümanların %60'ını bulabilmiş ve getirdiklerinin %80'i doğrudur.

Web aramasında ise geri çağırma çok anlamlı değildir; çünkü web'deki toplam ilgili doküman sayısını bilmek pratikte imkânsızdır. Bu nedenle web arama motorları daha çok sıralama kesinliği (rank precision) metriklerini kullanır: ilk 5, 10, 15, 20, 25 veya 30 sonuçtaki kesinlik değerleri hesaplanır; zira kullanıcılar nadiren 30 sonuçtan fazlasına bakarlar. F-skoru (F-measure), kesinlik ve geri çağırmanın harmonik ortalamasıdır ve tek bir sayısal değer sunar:

$$F = \frac{2 \cdot \text{Kesinlik} \cdot \text{Geri Çağırma}}{\text{Kesinlik} + \text{Geri Çağırma}} \quad (11)$$

Yukarıdaki örnekte $F = (2 \cdot 0.80 \cdot 0.60)/(0.80 + 0.60) = 0.96/1.40 \approx 0.686$ olur. F-skoru, iki metriğin dengesini tek bir sayıda ifade ettiği için sistem karşılaştırmalarında sıkça kullanılır.

2.10 IR ve Web Araması İlişkisi

Web araması, büyük ölçekli bir IR uygulamasıdır. Bir web tarayıcısı (crawler/robot) web'i dolaşarak sayfaları toplar. Toplanan sayfalar işlenir ve dev bir ters indeks veritabanı oluşturulur. Sorgu zamanında, arama motoru çeşitli vektör sorgu eşleştirme yöntemleri uygular. Farklı arama motorları arasındaki gerçek farklar şu noktalarda ortaya çıkar: indeks ağırlıklandırma şemaları (terimlerin konumu – başlık, gövde, vurgulu kelimeler vb.), sorgu işleme yöntemleri (sorgu

sınıflandırma, genişletme) ve sıralama algoritmaları. Bu bileşenlerin detayları, arama motoru şirketleri tarafından sıkı sıkıya korunan ticari sırlardır. Bununla birlikte, temeldeki IR prensipleri – TF-IDF ağırlıklandırması, ters indeksleme, vektör uzayı eşleştirmesi – tüm modern arama motorlarının omurgasını oluşturur.

3 Web Bağlantı Analizi, PageRank, HITS ve Merkezilik Ölçütleri

Web bağlantı analizi, 1996 yılından itibaren içerik benzerliği tabanlı arama motorlarının yetersiz kalmasıyla ortaya çıkmış bir araştırma alanıdır. İlk arama motorları TF-IDF ve kosinüs benzerliği gibi bilgi erişim yöntemlerini kullanarak sayfaları sıralarken, Web sayfalarının sayısının hızla artması ve içerik tabanlı yöntemlerin spam'e açık olması (sayfa sahiplerinin kelime tekrarlarıyla sıralamalarını yükseltebilmesi) araştırmacıları hiper-bağlantıları kullanmaya yöneltmiştir. 1997-1998 yıllarında rapor edilen en etkili iki hiper-bağlantı tabanlı arama algoritması, Sergey Brin ve Larry Page tarafından geliştirilen PageRank ile Jon Kleinberg tarafından geliştirilen HITS algoritmalarıdır. Her iki algoritma da sosyal ağ analizinden türetilmiş olup sayfaların “prestij” veya “otorite” seviyelerine göre sıralanmasını sağlar.

3.1 Sosyal Ağ Analizi ve Merkezilik Ölçütleri

Sosyal ağ analizi, bir organizasyon içindeki sosyal aktörler (insanlar) ve aralarındaki etkileşimlerin incelenmesidir. Her bir düğüm bir aktörü, her bir bağlantı ise bir ilişkiyi temsil eder. Web, esasen sanal bir toplum ve dolayısıyla sanal bir sosyal ağ olduğundan (her sayfa bir sosyal aktör, her hiper-bağlantı bir ilişkidir), sosyal ağ analizindeki birçok sonuç Web bağlamına uyarlanabilir. Bu bağlamda iki temel kavram öne çıkar: merkezilik (centrality) ve prestij (prestige). Merkezilik, bir aktörün diğer aktörlerle ne kadar kapsamlı bağlantı kurduğuna odaklanırken prestij, bir aktörün ne kadar “aranan” veya “referans gösterilen” olduğuna odaklanır.

3.1.1 Derece Merkeziliği (Degree Centrality)

Bir aktörün derece merkeziliği, sahip olduğu doğrudan bağlantıların sayısıdır. Yönlendirilmemiş bir grafta derece merkeziliği, düğümün toplam komşu sayısıdır; yönlendirilmiş bir grafta ise gelen bağlantı (indegree) ve giden bağlantı (outdegree) olarak ikiye ayrılır. Normalleştirme için derece merkeziliği, ağdaki mümkün olan maksimum bağlantı sayısı olan $(N - 1)$ değerine bölünür. Örneğin 5 düğümlü bir ağda 3 bağlantısı olan bir düğümün normalize derece merkeziliği $3/4 = 0.75$ olur. Freeman tarafından önerilen merkezileşme (centralization) formülü, ağdaki merkezilik skorlarının dağılımındaki asimetriyi ölçer. Bu formül, en yüksek merkezilik değerine sahip düğüm ile diğer tüm düğümler arasındaki normalize farkların toplamı olarak hesaplanır:

$$C_D = \frac{\sum_{i=1}^n [C_D(n^*) - C_D(i)]}{\max \sum_{i=1}^n [C_D(n^*) - C_D(i)]} \quad (12)$$

Burada $C_D(n^*)$ ağdaki en yüksek derece merkeziliği değeridir. Yüksek merkezileşme, birkaç düğümün çok sayıda bağlantıya sahip olduğu asimetric bir yapıyı gösterirken; düşük merkezileşme, bağlantıların daha eşit dağıldığını gösterir. Derece merkeziliğinin en büyük sınırlaması, bir düğümün ağ içindeki aracılık (brokerage) rolünü veya dolaylı etkisini yakalayamamasıdır. Bir düğüm az sayıda doğrudan bağlantıya sahip olsa bile, bu bağlantılar ağı farklı bölgelerini birbirine bağlayan kritik köprüler olabilir.

3.1.2 Yakınlık Merkeziliği (Closeness Centrality)

Yakınlık merkeziliği, bir düğümün ağdaki diğer tüm düğümlere olan ortalama en kısa yol uzaklığının tersidir. Bir düğüm diğer tüm düğümlere ne kadar yakınsa, bilgiye o kadar hızlı erişebilir ve ağdaki konumu o kadar merkezidir. Düğüm i için yakınlık merkeziliği aşağıdaki gibi tanımlanır:

$$C_C(i) = \frac{1}{\sum_{j \neq i} d(i, j)} \quad (13)$$

Normalleştirilmiş hali ise $(N - 1)$ ile çarpılarak elde edilir:

$$C'_C(i) = \frac{N - 1}{\sum_{j \neq i} d(i, j)} \quad (14)$$

Burada $d(i, j)$, i ile j arasındaki en kısa yol uzaklığıdır. Örneğin, 5 düğümlü bir ağda A düğümünün diğer düğümlere uzaklıkları sırasıyla 1, 1, 2, 3 ise yakınlık merkeziliği $1/(1 + 1 + 2 + 3) = 1/7 \approx 0.143$, normalize değeri ise $4/7 \approx 0.571$ olur. Yakınlık merkeziliği, bir düğümün aracı konumda olmasını gerektirmez; düğüm sadece diğerlerine “yakın” olmasıyla da yüksek merkezilik skoruna sahip olabilir.

3.1.3 Arasındalık Merkeziliği (Betweenness Centrality)

Arasındalık merkeziliği, bir düğümün diğer düğüm çiftleri arasındaki en kısa yollar üzerinde bulunma sıklığını ölçer. Eğer bir düğüm, diğer iki düğüm arasındaki etkileşimin zorunlu geçiş noktası ise, bu düğüm o etkileşim üzerinde kontrol sahibidir. Yönlendirilmemiş bir grafta, p_{jk} ile j ve k aktörleri arasındaki en kısa yol sayısı, $p_{jk}(i)$ ile de bu yollardan i 'den geçenlerin sayısı gösterilmek üzere, i düğümünün arasındalık merkeziliği şöyle tanımlanır:

$$C_B(i) = \sum_{j < k} \frac{p_{jk}(i)}{p_{jk}} \quad (15)$$

Normalleştirme için $(N - 1)(N - 2)/2$ değerine (mümkün olan maksimum düğüm çifti sayısı) bölünür. Örnek olarak, A–B–C–D–E şeklinde bir zincir yapı düşünelim. Düğüm C'nin arasındalık merkeziliğini hesaplayalım. A ile D arasındaki tek en kısa yol C'den geçer, dolayısıyla $p_{AD}(C)/p_{AD} = 1/1 = 1$. Benzer şekilde A ile E arasındaki tek yol da C'den geçer ve $p_{AE}(C)/p_{AE} = 1$. B ile D ve B ile E çiftleri için de durum aynıdır. C düğümü toplamda 4 düğüm çifti için tam kredi alır, yani normalize edilmemiş arasındalık değeri $C_B(C) = 4$ olur. Şimdi biraz daha karmaşık bir örnek ele alalım: A–B–C, A–D–C ve B–E–C bağlantılarına sahip bir grafta, A ile C arasında iki en kısa yol vardır (A–B–C ve A–D–C). Bu durumda B ve D düğümleri krediyi paylaşır, her biri $1/2$ kredi alır. Eğer bir düğüm tüm çiftler için toplam 1.5 kredi toplamışsa, bu onun normalize edilmemiş arasındalık değeridir.

3.1.4 Prestij Ölçütleri

Prestij, merkezilikten daha rafine bir öne çıkma ölçüsüdür ve gelen bağlantılara (in-links) odaklanır. Merkezilik giden bağlantıları vurgularken, prestij gelen bağlantıların sayısını ve kalitesini değerlendirir. Prestijli bir aktör, diğerlerinin yöneldiği, referans gösterdiği bir alıcı konumundadır.

Derece prestiji, en basit prestij ölçüsüdür ve bir düğüme gelen bağlantı sayısıdır. Yakınlık prestiji (proximity prestige), sadece doğrudan komşuları değil, düğüme ulaşabilen tüm aktörleri dikkate alır. I_i , i aktörüne ulaşabilen aktörlerin kümesi olmak üzere, yakınlık prestiji bu aktörlerin i 'ye olan ortalama uzaklığının tersiyle orantılıdır. $d(j, i)$ aktör j 'den aktör i 'ye olan yönlendirilmiş uzaklık ise:

$$P_P(i) = \frac{|I_i|/(N - 1)}{\sum_{j \in I_i} d(j, i)/|I_i|} \quad (16)$$

Sıra prestiji (rank prestige) ise en gelişmiş prestij ölçüsüdür ve PageRank ile HITS'in temelini oluşturur. Temel fikir, önemli bir aktör tarafından seçilen aktörün, daha az önemli bir aktör tarafından seçilenden daha prestijli olmasıdır. Sıra prestiji, gelen bağlantıların doğrusal bir kombinasyonu olarak tanımlanır. $P(i)$, i düğümünün sıra prestiji olmak üzere:

$$P(i) = \sum_j A_{ji} P(j) \quad (17)$$

Burada A , bitişiklik matrisidir. Bu denklem vektör formunda $P = A^T P$ olarak yazılır ki bu, PageRank denkleminin tam olarak aynıdır. P , A^T matrisinin özdeğeri 1 olan esas özvektörüdür.

3.2 PageRank Algoritması

PageRank, Web'in demokratik doğasına dayanır ve hiper-bağlantı yapısını bir sayfanın değerinin göstergesi olarak kullanır. Bir sayfadan diğerine verilen bağlantı, kaynak sayfanın hedef sayfa için verdiği bir "oy" olarak yorumlanır. Ancak PageRank sadece oyların sayısına değil, oy veren sayfanın kendi önemine de bakar: önemli sayfalardan gelen oylar daha değerli, daha az önemli sayfalardan gelen oylar ise daha az değerlidir. Bu, sıra prestiji fikrinin tam karşılığıdır.

3.2.1 Akış Formülasyonu (Flow Formulation)

Web'i yönlendirilmiş bir graf $G = (V, E)$ olarak modelliyoruz. Toplam sayfa sayısı n olsun. Her i sayfasının d_i adet giden bağlantısı (out-link) bulunsun. PageRank'in temel akış denklemi, bir sayfanın önem skorunun, kendisine bağlantı veren tüm sayfaların önem skorlarının, o sayfaların giden bağlantı sayılarına bölünmüş toplamı olduğunu söyler:

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i} \quad (18)$$

Burada $i \rightarrow j$ notasyonu, i 'den j 'ye bir bağlantı olduğunu ifade eder. Örneğin, üç sayfalı (y, a, m) basit bir Web grafi düşünelim: y sayfası y ve a'ya bağlantı versin ($d_y = 2$); a sayfası y ve m'ye bağlantı versin ($d_a = 2$); m sayfası sadece a'ya bağlantı versin ($d_m = 1$). Bu graf için akış denklemleri şöyle yazılır:

$$r_y = \frac{r_y}{2} + \frac{r_a}{2} \quad (19)$$

$$r_a = \frac{r_y}{2} + r_m \quad (20)$$

$$r_m = \frac{r_a}{2} \quad (21)$$

Bu üç denklem ve üç bilinmeyen vardır, ancak sabit terim yoktur. Tüm çözümler bir ölçek faktörüne göre denktir. Tek bir çözüm elde etmek için ek bir kısıt olarak tüm PageRank değerlerinin toplamının 1 olduğunu ($\sum r_i = 1$) varsayalım. Bu kısıt altında denklem sistemini çözelim. İkinci ve üçüncü denklemlerden $r_m = r_a/2$ elde edilir. Birinci denklemden $r_y/2 = r_a/2$, yani $r_y = r_a$ bulunur. Toplam kısıtı $r_y + r_a + r_m = 1$ olduğundan $r_a + r_a + r_a/2 = 1$, yani $5r_a/2 = 1$ ve $r_a = 2/5$ elde edilir. Sonuç olarak $r_y = 2/5$, $r_a = 2/5$, $r_m = 1/5$ bulunur.

3.2.2 Matris Formülasyonu

Akış denklemleri matris formunda ifade edilebilir. M matrisi, $M_{ji} = 1/d_i$ (eğer $i \rightarrow j$ bağlantısı varsa) ve aksi halde $M_{ji} = 0$ olarak tanımlanan sütun-stokastik bir matristir. Her sütunun toplamı 1'dir çünkü her sayfa toplam d_i giden bağlantıya sahiptir ve her birine öneminin $1/d_i$ 'sini aktarır. r , PageRank değerlerini tutan n boyutlu sütun vektörü olmak üzere:

$$r = M \cdot r \quad (22)$$

Bu, bir özdeğer problemidir: r , M matrisinin $\lambda = 1$ özdeğerine karşılık gelen esas özvektörüdür. M sütun-stokastik olduğu için en büyük özdeğeri 1'dir. Yukarıdaki 3 sayfalı örnek için M matrisi şöyledir:

$$M = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix} \quad (23)$$

Bu matris için $r = Mr$ denklemi $[2/5, 2/5, 1/5]^T$ çözümünü verir.

3.2.3 Güç İterasyonu (Power Iteration)

Büyük ölçekli Web grafları için Gauss eliminasyonu gibi doğrudan yöntemler pratik değildir. Bunun yerine güç iterasyonu adı verilen yinelemeli bir yöntem kullanılır. Algoritma şu adımlardan oluşur: İlk olarak tüm sayfalar için eşit başlangıç değerleri atanır, yani $r^{(0)} = [1/n, 1/n, \dots, 1/n]^T$. Ardından her iterasyonda $r^{(t+1)} = M \cdot r^{(t)}$ güncellemesi yapılır. İterasyon, $|r^{(t+1)} - r^{(t)}|_1 < \varepsilon$ koşulu sağlanana kadar devam eder (burada $|\cdot|_1$ L1 normudur, yani $|x|_1 = \sum |x_i|$, ve ε tipik olarak 10^{-6} gibi küçük bir eşik değeridir).

Üç sayfalı örneğimizde güç iterasyonunu adım adım uygulayalım. Başlangıç vektörü $r^{(0)} = [1/3, 1/3, 1/3]^T$ olsun. Birinci iterasyonda $r_y^{(1)} = 1/3 \cdot 1/2 + 1/3 \cdot 1/2 = 1/3$, $r_a^{(1)} = 1/3 \cdot 1/2 + 1/3 \cdot 1 = 1/2$, $r_m^{(1)} = 1/3 \cdot 1/2 = 1/6$ elde edilir, yani $r^{(1)} = [1/3, 1/2, 1/6]^T$. İkinci iterasyonda $r_y^{(2)} = 1/3 \cdot 1/2 + 1/2 \cdot 1/2 = 1/6 + 1/4 = 5/12$, $r_a^{(2)} = 1/3 \cdot 1/2 + 1/6 \cdot 1 = 1/6 + 1/6 = 1/3$, $r_m^{(2)} = 1/2 \cdot 1/2 = 1/4$ bulunur, yani $r^{(2)} = [5/12, 1/3, 1/4]^T$. İterasyonlar devam ettikçe vektör $[2/5, 2/5, 1/5]^T = [0.4, 0.4, 0.2]^T$ değerine yakınsar.

3.2.4 Rastgele Gezgin Modeli ve Markov Zinciri

PageRank, bir Markov zinciri olarak da yorumlanabilir. Her Web sayfası bir durum (state), her hiper-bağlantı ise bir durumdan diğerine geçiştir. Bir Web gezgininin sayfadaki tüm bağlantılar arasından rastgele ve eşit olasılıkla birini seçerek gezindiği varsayılır. $p(t)$, t anında gezginin her bir sayfada bulunma olasılığını gösteren sütun vektörü olsun. Bir sonraki adımdaki dağılım $p(t+1) = M \cdot p(t)$ ile hesaplanır. Markov zinciri teorisine göre, eğer geçiş matrisi stokastik, indirgenemez (irreducible) ve periyodik olmayan (aperiodic) ise, zincir başlangıç dağılımından bağımsız olarak tek bir durağan dağılıma yakınsar: $\lim_{k \rightarrow \infty} p_k = \pi$. Bu durağan dağılım $\pi = M^T \pi$ denklemini sağlar ve PageRank vektörü r olarak kullanılır. Bir sayfanın durağan dağılımdaki olasılığı ne kadar yüksekse, rastgele bir gezginin o sayfayı ziyaret etme olasılığı o kadar yüksektir, dolayısıyla sayfa o kadar prestijlidir.

Ancak gerçek Web grafi, bu üç koşulu da sağlamaz. Bu nedenle ideal durumdaki $r = Mr$ denklemi, “gerçek PageRank” modelini üretmek için genişletilmelidir.

3.2.5 Örümcek Tuzakları (Spider Traps)

Bir örümcek tuzağı, tüm giden bağlantıları kendi içindeki bir grup sayfaya yönlendiren ve dışarı çıkış imkanı vermeyen bir sayfa veya sayfa grubudur. Örümcek tuzakları, güç iterasyonunun tüm PageRank skorlarını bu tuzak içinde biriktirmesine ve diğer sayfaların skorlarının sıfıra gitmesine neden olur. İki sayfalı basit bir örnek düşünelim: a sayfası b'ye bağlantı versin, b sayfası sadece a'ya bağlantı versin. Bu bir örümcek tuzağıdır çünkü a ve b karşılıklı olarak birbirine bağlanmıştır ve dışarı çıkış yoktur. M matrisi, $M_{ba} = 1$ (a'nın tek giden bağlantısı b'ye), $M_{ab} = 1$ (b'nin tek giden bağlantısı a'ya) olur:

$$M = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad (24)$$

Başlangıç vektörü $r^{(0)} = [1, 0]^T$ ile başlarsak: $r^{(1)} = [0, 1]^T$, $r^{(2)} = [1, 0]^T$, $r^{(3)} = [0, 1]^T$ şeklinde salınım olur ve yakınsama sağlanamaz. Daha büyük bir örnekte ise tuzak tüm skoru emer. Örneğin m düğümünün kendisine döngü bağlantısı olduğu bir grupta ($m \rightarrow m$), m 'nin PageRank'i iterasyon ilerledikçe 1'e yaklaşır, diğerleri 0'a gider.

3.2.6 Çıkılmaz Sokaklar (Dead Ends)

Çıkılmaz sokak, hiç giden bağlantısı olmayan sayfalardır ($d_i = 0$). Bu tür sayfalar, matrisin ilgili sütununda tamamen sıfırlara neden olur ve M matrisi sütun-stokastik olma özelliğini kaybeder. Daha da önemlisi, çıkılmaz sokaklar önem skorlarının sistemden “sızmasına” ve tüm skorların zamanla sıfıra yakınsamasına yol açar. Örneğin, y sayfası a 'ya bağlantı versin, a sayfasının hiç giden bağlantısı olmasın. M matrisi şöyle olur:

$$M = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} \quad (25)$$

Başlangıç $r^{(0)} = [1, 0]^T$ ise $r^{(1)} = Mr^{(0)} = [0, 0]^T$ olur ve tüm skorlar anında sıfırlanır. Daha büyük graflarda çıkılmaz sokaklar her iterasyonda toplam skoru azaltır ve sonunda tüm skorlar sıfıra gider.

Çıkılmaz sokaklar için iki çözüm yaklaşımı vardır: birincisi, PageRank hesaplaması sırasında bu sayfaları tamamen çıkarmak; ikincisi ise her çıkılmaz sayfadan tüm Web sayfalarına yapay bağlantılar eklemektir. Pratikte ikinci yaklaşım tercih edilir.

3.2.7 Google'ın Çözümü: Rastgele Atlama Modeli

Google, hem örümcek tuzaklarını hem de çıkılmaz sokakları aynı stratejiyle çözer: her sayfadan diğer tüm sayfalara küçük bir geçiş olasılığı eklenir. Rastgele gezgin modelinde, gezgin her adımda iki seçeneğe sahiptir: β olasılıkla mevcut sayfadaki giden bağlantılardan birini rastgele seçerek takip eder (bu olasılık değerine sönmüleme faktörü veya damping factor denir; tipik değer $\beta = 0.85$ 'tir, yaygın aralık 0.8–0.9'dur); $1 - \beta$ olasılıkla ise Web'deki herhangi bir sayfaya rastgele atlar (teleport). Bu, “sıkılan gezgin” modeli olarak da bilinir; gezgin bir süre bağlantıları takip ettikten sonra sıkılır ve rastgele bir sayfaya gider. Google PageRank formülü şöyledir:

$$r_j = \beta \sum_{i \rightarrow j} \frac{r_i}{d_i} + (1 - \beta) \frac{1}{n} \quad (26)$$

Bu formülde ilk terim, bağlantı tabanlı oy aktarımını; ikinci terim ise teleport (rastgele atlama) bileşenini temsil eder. Her sayfa, teleport bileşeninden eşit miktarda $((1 - \beta)/n)$ pay alır. Vektör formunda bu denklem şöyle ifade edilir:

$$r = \beta Mr + (1 - \beta) \frac{1}{n} e \quad (27)$$

Burada e , tüm elemanları 1 olan n boyutlu sütun vektörüdür.

3.2.8 Google Matrisi

Google çözümü matris formunda daha derli toplu olarak ifade edilebilir. A , Google matrisi olarak adlandırılır ve şöyle tanımlanır:

$$A = \beta M + (1 - \beta) \frac{1}{n} ee^T \quad (28)$$

Burada ee^T , tüm elemanları 1 olan $n \times n$ boyutlu bir kare matristir. A matrisi üç kritik özelliğe de sahiptir: stokastiktir (çıkılmaz sokaklar için sütunlar $1/n$ ile doldurulduğu için her sütunun toplamı 1'dir), indirgenemezdir (her sayfadan diğerine sıfırdan büyük bir geçiş olasılığı vardır), ve periyodik değildir (teleport sayesinde her durumdan kendisine 1 adımda dönme olasılığı vardır, dolayısıyla $k = 1$ periyodu ile aperiodyktir). PageRank vektörü, A matrisinin esas özvektörüdür:

$$r = A \cdot r \quad (29)$$

Üç sayfalı (y, a, m) örümcek tuzağı içeren örneğimizde $\beta = 0.8$ ile Google matrisini oluşturalım. Orijinal M matrisinde m'nin kendisine döngüsü (spider trap) olduğunu varsayalım:

$$M = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 0 \\ 0 & 1/2 & 1 \end{bmatrix}, \quad \frac{1}{n}ee^T = \begin{bmatrix} 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \end{bmatrix} \quad (30)$$

$$A = 0.8 \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 0 \\ 0 & 1/2 & 1 \end{bmatrix} + 0.2 \begin{bmatrix} 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \end{bmatrix} = \begin{bmatrix} 7/15 & 7/15 & 1/15 \\ 7/15 & 1/15 & 1/15 \\ 1/15 & 7/15 & 13/15 \end{bmatrix} \quad (31)$$

Güç iterasyonu bu matrise uygulandığında, teleport sayesinde örümcek tuzağındaki m düğümü tüm skoru emmez. Kararlı durumda yaklaşık olarak $r = [7/33, 5/33, 21/33]^T$ değerine yakınsanır. Burada m hala en yüksek PageRank'e sahiptir (kendine döngü ve teleport kombinasyonuyla), ancak diğer sayfalar da sıfırlanmaz.

3.2.9 Markov Zinciri Koşullarının Detaylı İncelenmesi

PageRank'in matematiksel olarak doğru çalışması için geçiş matrisinin üç koşulu sağlaması gerekir. İlk olarak, **stokastiklik** koşulu her sütunun toplamının 1 olmasını gerektirir. Bu, A matrisindeki her sütundaki olasılıkların bir olasılık dağılımı oluşturmasını sağlar. Çıkamaz sokaklar bu koşulu bozar; teleport terimi $(1 - \beta)ee^T/n$ sayesinde her sütuna en azından küçük bir olasılık eklenerek stokastiklik sağlanır. Ayrıca çıkamaz sokak düğümlerinden tüm sayfalara $1/n$ olasılıkla teleport yapılması da alternatif bir çözümdür.

İkinci olarak, **indirgenemezlik** (irreducibility) koşulu, graftaki her düğüm çifti arasında yönlendirilmiş bir yol bulunmasını gerektirir. Başka bir deyişle, grafin kuvvetli bağlantılı (strongly connected) olması gerekir. Gerçek Web grafi, birçok düğüm çifti arasında yol bulunmadığı için bu koşulu sağlamaz. Örneğin, eski ve güncellenmeyen sayfalara hiçbir yerden bağlantı olmayabilir. Teleport, her sayfadan diğer tüm sayfalara doğrudan bir geçiş olasılığı ekleyerek grafi indirgenemez hale getirir. İndirgenemezlik olmazsa, Markov zinciri başlangıç durumuna bağlı olarak farklı durağan dağılımlara yakınsayabilir ve PageRank değerleri benzersiz olmaz.

Üçüncü olarak, **periyodik olmama** (aperiodicity) koşulu, bir durumdan kendisine dönüş yollarının uzunluklarının en büyük ortak böleninin 1 olmasını gerektirir. Eğer bir duruma dönüş yollarının uzunlukları hep k 'nın katıysa ($k > 1$), bu durum periyodiktir. Örneğin, üç düğümlü bir döngüde (1-2-3-1) periyot $k = 3$ 'tür. Periyodik zincirlerde güç iterasyonu yakınsamaz, salınım yapar. Teleport, her düğümden kendisine 1 adımda dönme olasılığı eklediği için $(1 - \beta)/n > 0$, periyodu otomatik olarak $k = 1$ yapar ve zinciri aperioidik hale getirir.

Bu üç koşul sağlandığında, Markov zinciri teorisi gereği zincirin tek ve pozitif bir durağan dağılımı vardır ve güç iterasyonu her başlangıç vektörü için bu dağılıma yakınsar.

3.3 HITS Algoritması

HITS (Hypertext Induced Topic Search), Jon Kleinberg tarafından 1998 yılında geliştirilmiş, sorguya bağımlı bir bağlantı analizi algoritmasıdır. PageRank'in aksine HITS, statik ve sorgudan bağımsız bir sıralama üretmez; kullanıcının sorgusuna göre dinamik olarak çalışır.

3.3.1 Otoriteler ve Göbekler (Authorities and Hubs)

HITS, Web sayfalarını iki role ayırır: otoriteler (authorities) ve göbekler (hubs). Bir otorite sayfası, belirli bir konuda güvenilir ve yetkin içeriğe sahip olan, bu nedenle birçok sayfa tarafından bağlantı verilen sayfadır. Bir göbek sayfası ise, bir konudaki otorite sayfalara işaret eden, bilgiyi organize eden bir "rehber" sayfadır. İyi bir göbek, birçok iyi otoriteye bağlantı verir; iyi bir otorite ise birçok iyi göbek tarafından işaret edilir. Bu iki rol arasında karşılıklı pekiştirme (mutual reinforcement) ilişkisi vardır.

3.3.2 Algoritmanın Çalışması

HITS, verilen bir q sorgusu için şu adımları izler: İlk olarak, q sorgusu bir arama motoruna gönderilir ve en yüksek sıralamaya sahip t sayfa toplanır ($t = 200$ tipik değerdir). Bu kümeye kök küme (root set) W denir. Ardından W kümesi genişletilir: W 'daki sayfaların bağlantı verdiği tüm sayfalar ve W 'daki sayfalara bağlantı veren sayfalar (belirli bir sınıra kadar, örneğin sayfa başına en fazla 50 bağlantı) eklenir. Böylece daha büyük bir taban küme (base set) S elde edilir. S kümesindeki sayfa sayısı n olsun ve L bu kümenin yönlendirilmiş bağlantı grafının ($G = (V, E)$) bitişiklik matrisi olsun: $L_{ij} = 1$ eğer i sayfası j 'ye bağlantı veriyorsa, aksi halde 0.

i sayfasının otorite skoru $a(i)$ ve göbek skoru $h(i)$ ile gösterilsin. Karşılıklı pekiştirme ilişkisi şu denklemlerle ifade edilir:

$$a(i) = \sum_{(j,i) \in E} h(j) \quad (32)$$

$$h(i) = \sum_{(i,j) \in E} a(j) \quad (33)$$

Yani bir sayfanın otorite skoru, kendisine bağlantı veren göbeklerin skorlarının toplamıdır; göbek skoru ise bağlantı verdiği otoritelerin skorlarının toplamıdır. Matris formunda:

$$a = L^T h \quad (34)$$

$$h = L a \quad (35)$$

Bu iki denklem birleştirildiğinde:

$$a = L^T (L a) = (L^T L) a \quad (36)$$

$$h = L (L^T h) = (L L^T) h \quad (37)$$

Otorite vektörü a , $L^T L$ matrisinin esas özvektörü; göbek vektörü h ise $L L^T$ matrisinin esas özvektörüdür. HITS skorları da PageRank'e benzer şekilde güç iterasyonu ile hesaplanır. Her iterasyonda vektörler normalize edilir. Başlangıçta tüm skorlar 1 olarak atanır, ardından sırasıyla $a = L^T h$, $h = L a$ güncellemeleri yapılır ve her adımda vektörler birim uzunluğa normalize edilir.

3.3.3 Eş-atıf ve Bibliyografik Eşleşme ile İlişkisi

HITS algoritmasının otorite ve göbek matrisleri, sırasıyla eş-atıf (co-citation) ve bibliyografik eşleşme (bibliographic coupling) kavramlarıyla doğrudan ilişkilidir. Eş-atıf matrisi C , iki sayfanın aynı üçüncü sayfalar tarafından ne sıklıkla birlikte referans gösterildiğini ölçer. L atıf matrisi olmak üzere ($L_{ij} = 1$ eğer i , j 'ye atıf yapıyorsa):

$$C = L^T L, \quad C_{ij} = \sum_{k=1}^n L_{ki} L_{kj} \quad (38)$$

C_{ij} değeri, hem i hem de j sayfalarına atıf yapan sayfaların sayısıdır. HITS'teki otorite matrisi tam olarak $L^T L$ 'dir, yani eş-atıf matrisidir.

Bibliyografik eşleşme matrisi B ise iki sayfanın aynı üçüncü sayfalara ne sıklıkla birlikte atıf yaptığını ölçer:

$$B = L L^T, \quad B_{ij} = \sum_{k=1}^n L_{ik} L_{jk} \quad (39)$$

B_{ij} değeri, hem i hem de j sayfalarının birlikte atıf yaptığı sayfaların sayısıdır. HITS'teki göbek matrisi tam olarak LL^T 'dir, yani bibliyografik eşleşme matrisidir.

Örnek olarak 4 sayfalı bir atıf ağı düşünelim: sayfa 1; sayfa 2 ve 3'e atıf yapsın, sayfa 2; sayfa 3'e atıf yapsın, sayfa 3 hiçbir yere atıf yapmasın, sayfa 4; sayfa 2 ve 3'e atıf yapsın. Bitişiklik matrisi L :

$$L = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix} \quad (40)$$

Eş-atıf matrisi $C = L^T L$:

$$C = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 3 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (41)$$

$C_{23} = 2$ olması, sayfa 2 ve sayfa 3'ün birlikte 2 sayfa (sayfa 1 ve sayfa 4) tarafından atıf aldığını gösterir. $C_{33} = 3$ ise sayfa 3'ün 3 farklı sayfa tarafından atıf aldığının öz-referansıdır.

3.3.4 HITS'in Güçlü ve Zayıf Yönleri

HITS'in en büyük avantajı, sorguya özgü otorite ve göbek sayfalarını belirleyebilmesidir. PageRank global ve sorgudan bağımsız bir sıralama üretirken, HITS kullanıcının aradığı konuya en uygun otoriteleri bulabilir. Ancak HITS'in üç önemli zayıflığı vardır. Birincisi, spam'e karşı PageRank'ten daha savunmasızdır; bir sayfa sahibi kendi sayfasına giden bağlantıları kontrol etmekte zorlanırken, kendi sayfasından giden bağlantıları kolayca ekleyebilir ve böylece göbek skorunu yapay olarak yükseltebilir. İkincisi, konu kaymasıdır (topic drift): genişletilmiş taban kümede sorgu konusuyla ilgisiz birçok sayfa bulunabilir ve bu sayfalar kendi aralarında yoğun bağlantılıysa HITS skorlarını domine edebilir. Üçüncüsü, sorgu zamanında verimsizdir; kök kümenin toplanması, genişletilmesi ve özvektör hesaplaması pahalı işlemlerdir.

3.4 Eş-atıf ve Bibliyografik Eşleşme Detayı

Eş-atıf ve bibliyografik eşleşme, bilimsel yayınların atıf analizinden doğmuş, ancak Web bağlamına da başarıyla uyarlanmış iki önemli kavramdır. Eş-atıf, iki sayfa i ve j 'nin her ikisine de atıf yapan (bağlantı veren) sayfaların sayısına dayanır. Ne kadar çok sayfa hem i 'ye hem j 'ye atıf yapıyorsa, i ve j arasındaki ilişki o kadar güçlüdür. Bibliyografik eşleşme ise tam tersi yönde çalışır: iki sayfa i ve j 'nin her ikisinin de atıf yaptığı ortak sayfaların sayısına dayanır. Bir başka deyişle eş-atıf, "kimler tarafından birlikte referans gösteriliyorlar" sorusuna; bibliyografik eşleşme ise "kimleri birlikte referans gösteriyorlar" sorusuna yanıt verir. Eş-atıf matrisi $C = L^T L$ simetriktir ve C_{ij} değeri, i ve j sütunlarının iç çarpımına eşittir. Bibliyografik eşleşme matrisi $B = LL^T$ de simetriktir ve B_{ij} değeri, i ve j satırlarının iç çarpımına eşittir. Bu iki matris, HITS'teki otorite ve göbek skorlarının hesaplanmasının matematiksel temelini oluşturur.

3.5 PageRank ve HITS'in Karşılaştırmalı Özeti

PageRank ve HITS, her ikisi de bağlantı analizi tabanlı olmalarına rağmen temel farklılıklar gösterir. PageRank sorgudan bağımsız, global bir ölçüdür ve tüm Web için çevrimdışı hesaplanır. Bu sayede sorgu zamanında hızlıdır, spam'e karşı dirençlidir ve Google'ın ticari başarısının temelini oluşturur. Ancak sorgudan bağımsız olması, genel olarak otoriter olan bir sayfa ile belirli bir sorgu konusunda otoriter olan bir sayfayı ayırt edememesine neden olur. HITS ise sorguya bağımlıdır, dinamik olarak çalışır ve hem otorite hem de göbek sıralaması üretir. Buna karşılık

spam'e daha açıktır, konu kayması yaşayabilir ve sorgu zamanında hesaplama maliyeti yüksektir. Modern arama motorları, hiper-bağlantı tabanlı sıralama algoritmalarını içerik tabanlı birçok faktörle birleştirerek kullanıcıya sunulan nihai sıralamayı oluşturur.

4 Web Crawler Mimarisi ve Bloom Filtreleri

Web madenciliğinin temel yapı taşlarından biri olan web crawler (tarayıcı), World Wide Web üzerindeki sayfaları otomatik olarak gezen ve bunları indirip analiz eden bir yazılım sistemidir. Crawler'lar; spider, robot, bot, web agent gibi farklı isimlerle de anılmakta olup Googlebot, Scooter, Slurp ve MSNBot gibi bilinen örnekleri mevcuttur. Bir crawler'ın temel amacı, başlangıç (seed) URL'lerinden yola çıkarak Web'i dolaşmak, sayfaları getirmek, bu sayfalardaki bağlantıları (link) çıkarmak ve bu bağlantıları tekrar ziyaret edilmek üzere bir kuyruğa eklemektir. Crawler'lar; evrensel arama motorlarını desteklemek, dikey (özel) arama motorları oluşturmak, iş zekası uygulamaları için rakipleri izlemek ve ilgi duyulan web sitelerini takip etmek gibi çok çeşitli amaçlarla kullanılmaktadır.

4.1 Crawler Taksonomisi

Crawler'lar en genel hatlarıyla iki ana kategoriye ayrılır: evrensel (universal) crawler'lar ve tercihli (preferential) crawler'lar. Evrensel crawler'lar Google, Yahoo ve Bing gibi genel amaçlı arama motorlarını desteklemek üzere tasarlanmıştır; milyarlarca sayfayı tarayacak şekilde ölçeklenmeleri gerekir ve tarama maliyetleri çok sayıda kullanıcı sorgusu üzerinden amorti edilir. Tercihli crawler'lar ise belirli bir kritere göre sayfaları seçici olarak ziyaret ederler. Tercihli crawler'lar kendi içinde odaklanmış (focused) ve konusal (topical) crawler'lar olarak ikiye ayrılır. Odaklanmış crawler'lar etiketlenmiş örnekler üzerinden eğitilmiş bir sınıflandırıcı kullanırken, konusal crawler'lar yalnızca bir konu tanımı ve başlangıç sayfaları ile çalışarak benzerlik tabanlı en iyi öncelikli arama (best-first search) gerçekleştirirler. Konusal crawler'ların altında adaptif crawler'lar, pekiştirmeli öğrenme tabanlı crawler'lar ve evrimsel algoritma tabanlı crawler'lar da bulunmaktadır.

4.2 Sıralı Crawler Mimarisi

Temel bir sıralı (sequential) crawler şu adımları izler: Başlangıç URL'lerinden oluşan seed kümesi bir frontier (sınır) veri yapısına yüklenir. Frontier'dan sıradaki URL çıkarılır, fetch işlemi ile sayfa indirilir, parse/extract aşamasında sayfanın içeriği çözümlenir ve bağlantılar çıkarılır, ardından sayfa repository'ye (depoya) kaydedilir. Çıkarılan yeni bağlantılar işlenerek frontier'a eklenir ve belirlenen durdurma kriteri sağlanana kadar döngü devam eder. Frontier, crawler'ın sayfaları hangi sırayla ziyaret edeceğini belirleyen kritik bir veri yapısıdır.

Frontier'ın gerçekleştirimi, Web'in bir çizge (graph) olarak taranması problemine karşılık gelir. İki temel çizge dolaşma stratejisi bulunur: Genişlik öncelikli arama (Breadth First Search, BFS) ve derinlik öncelikli arama (Depth First Search, DFS). BFS, bir kuyruk (queue, FIFO) ile gerçekleştirilir ve başlangıç sayfalarına en kısa yoldan ulaşılan sayfaları önceliklendirir. İyi seçilmiş seed sayfalarından başladığında BFS, crawler'ı kaliteli sayfalara yakın tutar. DFS ise bir yığın (stack, LIFO) ile gerçekleştirilir ve crawler'ın başlangıçtan hızla uzaklaşıp "siber uzayda kaybolmasına" (lost in cyberspace) yol açabilir. Bu nedenle web crawling'de BFS stratejisi tercih edilir.

4.3 URL İşleme

Crawler'ın karşılaştığı en önemli uygulama detaylarından biri URL'lerin doğru şekilde işlenmesidir. Sayfalardan çıkarılan bağlantılar genellikle göreceli (relative) URL'lerdir ve bunların mutlak (absolute) URL'lere dönüştürülmesi gerekir. Bu dönüşüm için HTTP başlığından, HTML meta etiketinden ya da varsayılan olarak mevcut sayfanın yolundan elde edilen bir temel (base) URL kullanılır. Örneğin, <http://www.cnn.com/linkto/> temel URL'sine sahip bir sayfada bulunan [intl.html](http://www.cnn.com/linkto/intl.html) göreceli URL'si <http://www.cnn.com/linkto/intl.html> mutlak URL'sine dönüştürülür. [/US/](http://www.cnn.com/US/) gibi kök dizinden başlayan göreceli bir URL ise <http://www.cnn.com/US/> şeklinde çözümlenir.

Aynı sayfaya işaret eden farklı URL'lerin yol açtığı tekrarlı taramayı önlemek amacıyla URL kanonikleştirmesi (canonicalization) uygulanır. Büyük-küçük harf farklılıkları (<http://WWW.CNN.COM/TECH/> yerine <http://www.cnn.com/TECH/>), varsayılan port numarası (:80 portunun kaldırılması), fragment tanımlayıcıları (`#fragment` kısmının atılması) ve `./.` ile `../` gibi yol kısaltmaları temizlenerek tüm URL'ler tek bir kanonik forma indirgenir. Ayrıca, `%7E` gibi yüzde kodlamalı karakterler karşılık gelen ASCII karakterine (örneğin `~`) dönüştürülür.

4.4 Metin İşleme

İndirilen sayfaların metin içeriği, dizinleme öncesinde bir dizi ön işleme adımından geçer. İlk adım, durdurma sözcüklerinin (stop words) çıkarılmasıdır. Anlam taşımayan, sözdizimsel işaretleyici niteliğindeki AND, THE, A, AT, OR, ON, FOR gibi en sık kullanılan sözcükler negatif bir sözlük (genellikle 10 ila 1000 öge) yardımıyla tespit edilir ve dizine eklenmeden elenir.

İkinci adım, birleştirme (conflation) işlemidir. Morfolojik birleştirmede, yüzeysel çekim farklılıkları göz ardı edilerek anlamsal olarak benzer sözcükler tek bir kök formda birleştirilir. Örneğin, {student, study, studying, studious} sözcükleri `studi` köküne indirgenir. Eş anlamlı sözcükler ise bir eş anlamlılar sözlüğü (thesaurus) kullanılarak birleştirilebilir; böylece kullanıcı “car” sorgusu yaptığında “automobile” içeren sayfalar da getirilebilir. Bu işlem indeks boyutunu %30–50 oranında küçültür.

Üçüncü adım, gövdeleme (stemming) işlemidir. Gövdeleme, dile bağlı yeniden yazım kurallarına dayanan morfolojik bir birleştirme tekniğidir. İngilizce için en yaygın kullanılan gövdeleme algoritması Porter Stemmer'dır. Porter Stemmer, bağlama duyarlı dilbilgisi kuralları kullanır; örneğin, “IES” eki “EIES” ya da “AIES” ile bitmiyorsa “Y” harfine dönüştürülür. Porter ayrıca, herhangi bir dilde gövdeleme algoritmaları oluşturmaya yarayan Snowball adlı bir dil geliştirmiştir. Gövdeleme algoritmaları Perl, C, Java, Python, C#, Ruby ve PHP gibi birçok dilde gerçekleştirilmiştir.

4.5 Örümcek Tuzakları

Crawler'ların karşılaştığı önemli sorunlardan biri örümcek tuzaklarıdır (spider traps). CGI betikleri tarafından dinamik olarak üretilen sonsuz sayıda sayfa ya da HTTP sunucusundaki yumuşak dizin bağlantıları ve yol yeniden yazma özellikleri kullanılarak keyfi derinlikte yollar oluşturulması, crawler'ın sonsuz bir URL uzayında sıkışıp kalmasına neden olabilir. Bu tuzaklara karşı yalnızca sezgisel (heuristic) savunma önlemleri alınabilir: URL uzunluğunun belirli bir eşiğin (örneğin 128 karakter) üzerinde olması durumunda bunun bir tuzak olduğu varsayılır, bir alan adından çok sayıda URL gelmesi izlenir, metinsel olmayan veri türlerine sahip URL'ler elenir ve mümkünse dinamik sayfaların taranması devre dışı bırakılır.

4.6 Sayfa Deposu

İndirilen sayfaların depolanması (page repository) için naif yaklaşım her sayfanın ayrı bir dosya olarak saklanmasıdır. Bu durumda URL'den benzersiz bir dosya adı üretmek için MD5 gibi bir özetleme (hashing) fonksiyonu kullanılabilir. Ancak bu yöntem çok sayıda küçük dosya oluşturduğundan depolama açısından verimsizdir. Daha iyi bir yaklaşım, birçok sayfanın XML işaretleme ile ayrılarak tek bir büyük dosyada birleştirilmesidir; bu durumda her sayfa için {dosya adı, sayfa kimliği} eşlemesi tutulur. İndeksleme için ilişkisel veritabanı yönetim sistemleri büyük ek yük getirdiğinden, Berkeley DB gibi hafif ve gömülü veritabanları tercih edilir.

4.7 Eş Zamanlılık

Bir crawler üç temel gecikme ile karşı karşıyadır: URL'deki sunucu adının IP adresine çözümlenmesi için DNS sorgusu, sunucuya socket bağlantısı açılıp isteğin gönderilmesi ve istenen sayfanın yanıt olarak alınması. Bu gecikmelerin üst üste bindirilerek toplam tarama

süresinin azaltılması için eş zamanlı (concurrent) crawler mimarisi kullanılır. Çoklu işlem (multi-processing) ya da çoklu iş parçacığı (multi-threading) ile 5–10 kat hızlanma elde etmek mümkündür; ancak milyarlarca sayfa ölçeğinde bu yaklaşımlar yetersiz kalır.

Büyük ölçekli evrensel crawler’larda engellemesiz (non-blocking) I/O kullanılır: aynı anda yüzlerce ağ bağlantısı açık tutulur ve soketler yoklanarak (polling) ağ transferlerinin tamamlanması izlenir. DNS çözümlemeleri için TCP yerine daha az ek yük getiren UDP protokolü kullanılır ve büyük, kalıcı bir bellek içi DNS önbelleği ile önceden getirme (prefetching) yapılır. Yükü sunuculara yaymak için birden çok paralel kuyruk kullanılır ve bağlantılar canlı tutulur. Tüm bu bileşenler, optimize edilmiş ağ bant genişliği ve disk G/Ç çıktısı ile devasa bir tarama makinesi çiftliğinde (crawl machine farm) çalışır.

4.8 Tazelik Politikaları

Evrensel crawler’lar hiçbir zaman “tamamlanmaz”; Web’deki sayfalar sürekli değişir, eklenir ve silinir. HTTP başlıklarındaki Last-Modified ve Expires alanları sayfa değişimlerini izlemek için güvenilir değildir. Bu nedenle, daha önce ziyaret edilmiş bir sayfanın o zamandan beri değişmiş olma olasılığını tahmin eden ve bu olasılığa göre önceliklendirme yapan tazelik (freshness) politikaları geliştirilmiştir. Brewington ve Cybenko ile Cho, Garcia-Molina ve Page tarafından önerilen algoritmalar, çoğu sayfanın belirli bir eşikten daha taze olmasını garanti edecek şekilde tarama planlaması yapar. Ntoulas, Cho ve Olston (2004) tarafından yapılan çalışmada, yakın geçmişin geleceği tahmin edeceği varsayımı altında, değişim sıklığının (change frequency) değil, değişim derecesinin (degree of change) daha iyi bir tahminleyici olduğu gösterilmiştir.

4.9 Gelişmiş Odaklanmış Tarama Algoritmaları

Konusal crawler’larda frontier, sayfaların konuyla benzerliğine göre sıralanmış bir öncelik kuyruğu olarak gerçekleştirilir. En basit konusal crawler olan Naïve Best-First, her adımda frontier’daki en yüksek benzerlik puanına sahip bağlantıyı çeker, sayfayı indirir, konuyla benzerliğini hesaplar ve çıkan bağlantıları ebeveyn sayfanın benzerlik puanıyla birlikte frontier’a ekler.

Best-N-First algoritması, frontier’ı her bağlantı eklendiğinde yeniden sıralamak yerine yalnızca her N sayfa ziyaret edildiğinde tembel bir sıralama (lazy sort) yapar. Bu yaklaşım, daha az açgözlü davranarak keşif (exploration) ile sömürü (exploitation) arasında daha iyi bir denge kurar ve crawler’ın yerel konusal tuzaklardan kurtulmasını sağlayarak performansı önemli ölçüde artırır.

Shark Search algoritması, Fish Search’ün daha güçlü bir türevidir ve frontier öncelik kuyruğunu sıralarken yalnızca benzerlik puanını değil; bağlantı metnini (anchor text), ebeveyn sayfanın benzerliğini ve ebeveynin kalitesini bir arada değerlendiren birleşik bir puanlama kullanır.

Web’in konusal yerelliği (topical locality), konusal crawler’ların çalışabilmesi için hem gerekli hem de yeterli bir koşuldur; bağlantılar rastgele değil, anlamsal bilgi taşımalıdır. Bu yerellikten yararlanan iki önemli ipucu mevcuttur. Ortak atıf (co-citation, sibling locality), A ve C sayfaları iyi birer hub ise A ve D sayfalarına da yüksek öncelik verilmesi gerektiğini söyler. Ortak referans (co-reference, bibliographic coupling) ise, E ve G sayfaları iyi birer otorite ise E ve H sayfalarına da yüksek öncelik verilmesi gerektiğini belirtir. Bu iki mekanizma, sırasıyla hub yerelliği ve otorite yerelliği olarak da adlandırılır.

4.10 Bloom Filtreleri

Web crawler bağlamında Bloom filtrelerinin temel motivasyonu, URL tekilleştirme (deduplication) problemidir. Bir crawler, şimdiye kadar keşfettiği tüm URL’leri merkezi bir listede tutmak ve yeni gelen URL’lerin daha önce görülüp görülmediğini hızlıca kontrol etmek zorundadır. Milyarlarca URL söz konusu olduğunda, ziyaret edilmiş URL’lerin tam listesini bellekte tutmak bellek kısıtı nedeniyle mümkün değildir. Bloom filtresi, az miktarda bellek kullanarak

“bu URL kesinlikle daha önce görülmedi” ya da “bu URL muhtemelen daha önce görüldü” şeklinde yanıt veren, olasılıksal bir veri yapısıdır.

4.10.1 Veri Yapısı ve Çalışma Prensipleri

Bir Bloom filtresi, başlangıçta tüm bitleri 0 olan N bitlik bir dizi ve k adet bağımsız özetleme (hash) fonksiyonundan oluşur. Her bir hash fonksiyonu h_i , bir URL’yi (ya da genel olarak bir akış elemanını) alır ve $\{0, 1, \dots, N - 1\}$ aralığında bir dizi indisi döndürür.

Ekleme (insertion) işlemi şu şekilde gerçekleşir: Sisteme yeni bir x elemanı geldiğinde, k hash fonksiyonunun her biri için $h_i(x)$ indisindeki bit 1 yapılır. Sorgulama (lookup) işlemi ise, bir y elemanının daha önce eklenip eklenmediğini kontrol etmek için tüm $h_i(y)$ indislerine bakar: Eğer bu indislerdeki bitlerin tamamı 1 ise filtre “evet, y daha önce görülmüş olabilir” der; en az bir tanesi 0 ise filtre “hayır, y kesinlikle daha önce görülmedi” yanıtını verir. Buradaki kritik özellik, Bloom filtresinin yanlış negatif (false negative) üretmemesidir — bir eleman daha önce eklenmişse filtre mutlaka onu tanır. Ancak yanlış pozitif (false positive) üretmesi mümkündür; daha önce eklenmemiş bir eleman için, hash çakışmaları nedeniyle tüm bitlerin tesadüfen 1 olması durumunda filtre yanlışlıkla “evet” diyebilir.

4.10.2 Olasılıksal Analiz: Dart Atma Modeli

Bloom filtresindeki yanlış pozitif olasılığını analiz etmek için dart atma (throwing darts) modeli kullanılır. t adet hedefe (bit pozisyonu) rastgele d adet dart atıldığını düşünelim. Burada d , eklenen toplam dart sayısıdır ve m eleman eklenmişse $d = k \cdot m$ olur.

Tek bir dartın belirli bir hedefi vurma olasılığı $1/t$ ’dir. Buna göre, d dartın hiçbirinin belirli bir hedefi vurmama olasılığı aşağıdaki gibi hesaplanır:

$$P(\text{bit 0 kalır}) = \left(1 - \frac{1}{t}\right)^d \quad (42)$$

Bu ifade, t büyük olduğunda üstel fonksiyona yakınsar:

$$\left(1 - \frac{1}{t}\right)^d = \left[\left(1 - \frac{1}{t}\right)^t\right]^{d/t} \approx e^{-d/t} \quad (43)$$

Dolayısıyla dizideki bitlerin 0 kalma oranı yaklaşık olarak $e^{-d/t}$, 1 olma yoğunluğu ise $1 - e^{-d/t}$ olur.

4.10.3 Yanlış Pozitif Olasılığı

Bir yanlış pozitif olayı, daha önce eklenmemiş bir y elemanı için k hash fonksiyonunun tamamının hâlihazırda 1 olan bitlere işaret etmesi durumunda gerçekleşir. Her bir hash sonucunun 1 olan bir bite denk gelme olasılığı, dizideki 1’lerin yoğunluğuna eşittir. k bağımsız hash fonksiyonu için yanlış pozitif olasılığı:

$$P(\text{FP}) = \left(1 - e^{-d/t}\right)^k = \left(1 - e^{-km/t}\right)^k \quad (44)$$

4.10.4 Sayısal Örnek

Bloom filtresinin çalışmasını somut bir örnek üzerinden inceleyelim. $t = 10^9$ bitlik (yaklaşık 125 MB) bir dizi, $k = 5$ hash fonksiyonu kullandığımızı ve $m = 10^8$ adet URL (100 milyon) eklediğimizi varsayalım. Toplam atılan dart sayısı $d = k \cdot m = 5 \times 10^8$ olur. Bu durumda $d/t = 5 \times 10^8 / 10^9 = 0.5$ oranı elde edilir. 0 kalma oranı:

$$e^{-d/t} = e^{-0.5} \approx 0.607 \quad (45)$$

Dizideki bitlerin yaklaşık %60.7'si 0 olarak kalırken, 1'lerin yoğunluğu:

$$1 \text{ yoğunluğu} = 1 - 0.607 = 0.393 \quad (46)$$

Bu durumda yanlış pozitif olasılığı:

$$P(\text{FP}) = (0.393)^5 \approx 0.00937 \approx \%0.94 \quad (47)$$

Görüldüğü gibi, 125 MB civarında bir bellek kullanarak 100 milyon URL için yanlış pozitif oranı %1'in altına düşürülebilmektedir. Bu, tam bir hash tablosunun gerektireceği gigabaytlar seviyesindeki belleğe kıyasla son derece verimli bir çözümdür. Yanlış pozitif durumunda olabilecek en kötü senaryo, crawler'ın daha önce ziyaret ettiği bir URL'yi ziyaret edilmedi sanıp tekrar fetch etmeye çalışmasıdır; bu da yalnızca küçük bir bant genişliği ve zaman kaybına yol açar. Buna karşılık yanlış negatif söz konusu olmadığından, gerçekten yeni olan hiçbir URL kaçırılmaz.

5 Web Kullanım Madenciliği

5.1 Giriş ve Temel Kavramlar

Web kullanım madenciliği (Web Usage Mining), bir veya birden fazla web sitesiyle kullanıcı etkileşimleri sonucunda toplanan veya üretilen tıklama akışı (clickstream) ve ilişkili verilerden otomatik olarak örüntü keşfetme sürecidir. Temel amaç, bir web sitesiyle etkileşime giren kullanıcıların davranışsal örüntülerini ve profillerini analiz etmektir. Keşfedilen örüntüler genellikle, ortak ilgi alanlarına sahip kullanıcı grupları tarafından sıklıkla erişilen sayfa, nesne veya kaynak koleksiyonları olarak temsil edilir. Web kullanım madenciliği, geleneksel olarak e-ticaret sitelerinin organizasyonunu iyileştirmek ve kârı artırmak için kullanılmış olsa da günümüzde arama motorlarının sonuç kalitesini değerlendirmesinden kişiselleştirme uygulamalarına kadar geniş bir yelpazede kullanılmaktadır.

5.2 Üç Aşamalı Süreç

Web kullanım madenciliği süreci üç ana aşamadan oluşur: Veri Ön İşleme (Data Preprocessing), Örüntü Keşfi (Pattern Discovery) ve Örüntü Analizi (Pattern Analysis). Veri ön işleme aşamasında ham sunucu günlükleri temizlenir, kullanıcılar tanımlanır, oturumlar oluşturulur (sessionization), önbellek kaynaklı eksik referanslar tamamlanır (path completion) ve sayfa görüntülemeleri (pageview) belirlenir. Bu aşama, ham veriden anlamlı işlem verisine geçişin temelidir ve genellikle tüm sürecin en zahmetli kısmını oluşturur. Örüntü keşfi aşamasında, hazırlanan oturum verisine sık öge kümeleri, birliktelik kuralları, ardışık örüntüler, kümeleme ve sınıflandırma gibi veri madenciliği algoritmaları uygulanır. Örüntü analizi aşamasında ise keşfedilen örüntüler iş değeri açısından yorumlanır; anlamlı olmayan kurallar elenir ve eyleme dönüştürülebilir içgörüler çıkarılır.

5.3 Veri Kaynakları

Web kullanım madenciliğinde dört temel veri kaynağı vardır. Bunlardan ilki ve en önemlisi sunucu günlükleridir (server logs). Her HTTP isteği, tipik olarak istemci IP adresi, zaman damgası, HTTP metodu, istenen kaynağın URL'si, HTTP durum kodu, yanıt boyutu ve kullanıcı aracı (user agent) bilgilerini içeren bir satır olarak kaydedilir. Örneğin tipik bir günlük satırı şu bilgileri taşır: 2006-02-01 00:08:43 1.2.3.4 - GET /classes/cs589/papers.html - 200 9221 Mozilla/4.0 İkinci veri kaynağı site içeriği ve yapısıdır; HTML sayfalarının içerdiği metinler, bağlantı yapıları ve sayfa türleri, kullanım verisini zenginleştirmek için kullanılır. Üçüncü kaynak, harici kanallardan toplanan demografik ve kullanıcı profili verileridir; kullanıcıların yaş, cinsiyet, konum gibi öznitelikleri bu kapsamdadır. Dördüncü kaynak ise e-ticaret verileridir; ürün görüntüleme, sepete ekleme ve satın alma gibi işlem olaylarını içerir. Bu verilerin tümü her zaman mevcut olmayabilir ve mevcut olduğunda başarıyla entegre edilmeleri gerekir; bu da web kullanım madenciliğinin en büyük zorluklarından biridir.

5.4 Kullanıcı Tanımlama Yöntemleri

Bir web sitesini ziyaret eden farklı kullanıcıları ayırt etmek, web kullanım madenciliğinin en kritik ve en zor ön işleme adımlarındandır. Proxy sunucuları, anonimleştiriciler, dinamik IP adresleri ve paylaşımlı bilgisayarlar nedeniyle birebir kullanıcı tanımlaması her zaman mümkün olmaz. Bu amaçla dört temel yöntem kullanılır.

IP adresi ve kullanıcı aracı (User Agent) kombinasyonuna dayalı yöntem en basit ve en yaygın yaklaşımdır. Aynı IP adresinden ve aynı tarayıcı bilgisinden gelen istekler aynı kullanıcıya atfedilir. Bu yöntem hiçbir ek altyapı gerektirmez ve tamamen sunucu günlüklerinden çalışır; kurulum maliyeti sıfırdır. Ancak ciddi dezavantajları vardır: NAT arkasındaki birden fazla kullanıcı aynı IP'yi paylaştığında tek kullanıcı gibi görünür, dinamik IP kullanan bir kullanıcı

oturumlar arasında farklı IP'lerle görünebilir ve tarayıcı bilgisi kolayca taklit edilebilir. Gizlilik düzeyi düşüktür çünkü kullanıcının IP adresi kaydedilir.

Gömülü oturum tanımlayıcısı (Embedded Session ID), her sayfa URL'sine dinamik olarak eklenen benzersiz bir kimliktir. Kullanıcı siteye girdiğinde sunucu tarafından üretilir ve tıklanan her bağlantıya otomatik olarak eklenir. Avantajı, IP değişse bile oturumun takip edilebilmesidir; dezavantajı ise URL'lerin paylaşılması durumunda oturum kimliğinin de paylaşılması ve kullanıcının yer imlerine eklediği sayfalarda eski oturum kimliklerinin kalmasıdır. Gizlilik açısından IP yöntemine benzerdir; kullanıcıya özel kalıcı bir kimlik içermez.

Kullanıcı kaydı (Registration) yöntemi, kullanıcının siteye kullanıcı adı ve parola ile giriş yapmasını gerektirir. Bu yöntem en güvenilir kullanıcı tanımlamasını sağlar; farklı cihazlardan ve farklı IP'lerden gelen aynı kullanıcı doğru şekilde ilişkilendirilir. Ayrıca demografik verilerle entegrasyon da mümkündür. Buna karşılık en büyük dezavantajı, kullanıcıların büyük çoğunluğunun kayıt olmadan siteyi terk etmesi nedeniyle verinin çok küçük bir kısmını kapsamasıdır. Gizlilik düzeyi, kullanıcının kimliğini açıkça ifşa etmesi nedeniyle en düşük olanıdır.

Çerez (Cookie) tabanlı yöntem, kullanıcının tarayıcısına küçük bir metin dosyası yerleştirerek çalışır. Bu çerez, benzersiz bir kullanıcı kimliği içerir ve tarayıcı tarafından sonraki tüm isteklerde sunucuya geri gönderilir. En büyük avantajı, kayıt gerektirmemesi ve kullanıcının IP'si değişse bile tutarlı takip sağlamasıdır. Ayrıca kullanıcıya özel ayarların saklanmasına da olanak tanır. Dezavantajları ise kullanıcının çerezleri silebilmesi veya devre dışı bırakabilmesi, farklı tarayıcıların farklı çerezler tutması ve paylaşımlı bilgisayarlarda birden fazla kullanıcının aynı çerezi kullanmasıdır. Gizlilik endişeleri nedeniyle birçok ülkede çerez kullanımı yasal düzenlemelere tabidir; kullanıcıya bilgi verilmesi ve onay alınması gerekebilir.

Yazılım aracı (Software Agent), kullanıcının bilgisayarına yüklenen özel bir yazılımın tüm tarama aktivitelerini kaydetmesi esasına dayanır. En doğru ve en eksiksiz veriyi sağlar; yalnızca belirli bir siteyi değil, kullanıcının tüm web aktivitelerini izleyebilir, sayfada geçirilen süreyi tam olarak ölçebilir ve önbellek sorunundan etkilenmez. Buna karşılık, kullanıcının yazılımı gönüllü olarak yüklemesini gerektirir, bu da örneklemin çok küçük ve yanlı olmasına yol açar. Ayrıca gizlilik açısından en sorunlu yöntemdir; kullanıcının yalnızca site içi değil site dışı aktiviteleri de kaydedilebilir, bu da ciddi etik ve yasal sorunlar doğurur.

5.5 Oturumlara Ayırma (Sessionization)

Kullanıcı tanımlandıktan sonra, aynı kullanıcıya ait isteklerin anlamlı oturumlara bölünmesi gerekir. Bir oturum (session), kullanıcının siteye girdiği andan çıktığı ana kadar gerçekleştirdiği tüm aktivitelerin bütünüdür. Oturum sınırlarını belirlemek için iki temel strateji ailesi kullanılır: zaman odaklı sezgiseller (time-oriented heuristics) ve navigasyon odaklı sezgiseller (navigation-oriented heuristics).

Zaman odaklı sezgiseller iki yaygın kural içerir. Birinci sezgisel (h1), toplam oturum süresini sınırlar: bir kullanıcının ilk ve son isteği arasındaki süre 30 dakikayı aşarsa, oturum sonlandırılır ve yeni bir oturum başlatılır. Bu yaklaşım, tipik bir web oturumunun yarım saatten uzun sürmeyeceği varsayımına dayanır. İkinci sezgisel (h2), sayfada kalma süresini sınırlar: ardışık iki istek arasındaki süre 10 dakikayı aşarsa, bir oturum sınırı varsayılır ve yeni bir oturum başlatılır. Bu kural, kullanıcının bir sayfaya bakıp 10 dakikadan uzun süre hareketsiz kalması durumunda, büyük olasılıkla başka bir işle meşgul olduğu veya siteyi terk ettiği varsayımına dayanır. Her iki sezgisel de genellikle birlikte kullanılır; örneğin 30 dakikalık toplam süre sınırı içinde, 10 dakikalık sayfa kalma süresi aşımaları da oturum sınırı olarak değerlendirilir.

Navigasyon odaklı sezgiseller, istekler arasındaki bağlantı yapısını kullanır. Temel fikir şudur: eğer bir sayfadan yapılan isteğin referrer (yönlendiren) alanı, site içi bağlantı grafiğinde mevcut bir bağlantıya işaret etmiyorsa, bu istek büyük olasılıkla yeni bir oturumun başlangıcıdır. Örneğin bir kullanıcı Google'dan doğrudan bir ürün sayfasına geliyorsa ve bu sayfa önceki sayfadan bağlantı ile erişilebilir değilse, yeni bir oturum başlatılır. Navigasyon sezgisellerine zaman boyutu da eklenir: eğer referrer doğrulaması başarısız olursa ve sayfada kalma süresi 10 saniyenin

altındaysa, bu istek bir önceki oturuma dahil edilir; çünkü bu kadar kısa bir süre, kullanıcının hızlı bir geri dönüş yaptığına işaret eder. Navigasyon odaklı yöntemler, özellikle frame tabanlı sitelerde ve AJAX uygulamalarında daha karmaşık hale gelir.

5.6 Önbellek Etkisi ve Yol Tamamlama (Path Completion)

Web tarayıcılarının ve proxy sunucularının kullandığı önbellekleme (caching) mekanizması, sunucu günlüklerinde önemli veri kayıplarına yol açar. Bir kullanıcı aynı oturum içinde daha önce ziyaret ettiği bir A sayfasına geri dönerse, tarayıcı sayfayı önbellekten yükler ve sunucuya yeni bir istek göndermez. Sonuç olarak, A sayfasına yapılan bu ikinci erişim sunucu günlüklerinde görünmez. Bu durum, kullanıcının tarayıcıdaki geri düğmesini (back button) kullanmasıyla çok sık gerçekleşir. Örneğin bir kullanıcı sırasıyla Ana Sayfa → Ürünler → Ürün A sayfalarını ziyaret etsin. Daha sonra geri düğmesiyle Ürünler sayfasına dönsün ve buradan Ürün B sayfasına gitsin. Sunucu günlüklerinde Ana Sayfa, Ürünler, Ürün A ve Ürün B istekleri görünür; ancak Ürünler sayfasına ikinci erişim kaydedilmez. Günlüklerdeki sıralama Ana Sayfa → Ürünler → Ürün A → Ürün B şeklinde olur ve Ürün A'dan Ürün B'ye doğrudan bir bağlantı varmış gibi yanıltıcı bir gezinme yolu oluşur.

Yol tamamlama (path completion), bu eksik referansları site bağlantı grafiğini kullanarak yeniden oluşturma işlemidir. Algoritma, site içindeki tüm sayfalar arasındaki bağlantı yapısını bir graf olarak modeller. Ardışık iki günlük kaydı arasında, graf üzerinden en kısa yol (veya en olası yol) hesaplanarak aradaki eksik sayfalar tamamlanır. Örneğin Ürün A sayfasından Ürün B sayfasına doğrudan bir bağlantı yoksa, ancak Ürün A'dan Ürünler sayfasına ve Ürünler sayfasından Ürün B sayfasına bağlantı varsa, yol tamamlama algoritması araya Ürünler sayfasını ekler. Günlüklerdeki referrer bilgisi de bu süreçte belirsizliği gidermek için kullanılır; eğer referrer alanı eksik gezinme yolunu doğruluyorsa, yol tamamlama daha güvenilir hale gelir. Frame tabanlı sitelerde, bir sayfa görüntülemesi birden fazla HTTP isteğinden oluştuğu için yol tamamlama çok daha karmaşılaşır ve her frame için ayrı bir bağlantı grafiği gerekebilir.

5.7 Sayfa Görüntüleme (Pageview)

Sayfa görüntüleme (pageview), tek bir kullanıcı eylemi (örneğin bir tıklama) sonucunda tarayıcıda görüntülenen web nesnelerinin toplamıdır. Kavramsal olarak her sayfa görüntülemesi, belirli bir kullanıcı olayını (event) temsil eden bir nesne koleksiyonu olarak düşünülebilir: bir makale okumak, bir ürün sayfasını görüntülemek veya alışveriş sepetine ürün eklemek gibi. Teknik olarak bir HTML sayfasının yüklenmesi birden fazla HTTP isteği üretir: ana HTML dosyası, CSS stil dosyaları, JavaScript dosyaları, resimler ve diğer gömülü nesneler. Veri ön işleme sırasında bu yardımcı istekler ana sayfa isteği altında toplanarak tek bir sayfa görüntülemesi oluşturulur. Bu işlem, analizin odağını ham HTTP isteklerinden anlamlı kullanıcı eylemlerine kaydırır.

5.8 E-Ticaret Verisi ile Entegrasyon

E-ticaret sitelerinde web kullanım madenciliği, kullanıcıların gezinti (browsing) davranışından satın alma davranışına dönüşümünü (conversion) analiz etmek için kritik öneme sahiptir. Bu entegrasyon iki boyutta ele alınır: ürün odaklı olaylar ve sayfa türü sınıflandırması.

Ürün odaklı olaylar dört ana kategoride toplanır. Ürün Görüntüleme (Product View), bir ürünün herhangi bir sayfa görüntülemesinde gösterilmesiyle tetiklenir; görüntüleme türü olarak resim (Image), bağlantı (Link) veya metin (Text) olarak sınıflandırılabilir. Ürün Tıklaması (Product Click-through), kullanıcının bir ürüne daha fazla bilgi almak için tıklamasıyla oluşur. Alışveriş Sepeti Değişiklikleri (Shopping Cart Changes), sepete ürün ekleme, sepetten ürün çıkarma veya sepetteki bir ürünün miktar ya da beden gibi özelliklerinin değiştirilmesi olaylarını kapsar. Ürün Satın Alma veya Teklif Verme (Product Buy or Bid), sepetteki her ürün için

ayrı bir satın alma olayı olarak kaydedilir; açık artırma sitelerinde satın almanın yanı sıra teklif verme (bid) olayları da izlenir.

Sayfa türü sınıflandırması, sitenin sayfalarını dört işlevsel kategoriye ayırır. Başlık sayfaları (Head pages), kullanıcıyı siteye yönlendiren giriş sayfalarıdır; genellikle ana sayfa veya kategori ana sayfaları bu türdendir. Medya sayfaları (Media pages), asıl içeriği barındıran sayfalar; ürün detay sayfası, makale sayfası veya blog yazısı buna örnektir. Navigasyon sayfaları (Navigation pages), kullanıcıyı diğer sayfalara yönlendirmek için tasarlanmıştır; site haritası, kategori listeleri ve arama sonuç sayfaları bu gruba girer. Arama ve veri girişi sayfaları (Look-up/Data Entry pages), kullanıcının form doldurduğu, arama yaptığı veya hesap bilgilerini girdiği sayfalardır. Bu sınıflandırma, oturum içindeki sayfa geçişlerinin daha anlamlı analiz edilmesini sağlar; örneğin bir kullanıcının navigasyon sayfasından medya sayfasına geçişi, ürün keşfetme davranışı olarak yorumlanabilir.

5.9 Örüntü Keşfi için Madencilik Teknikleri

Veri ön işleme tamamlandıktan ve oturumlar oluşturulduktan sonra, çeşitli veri madenciliği algoritmaları uygulanarak kullanıcı davranış örüntüleri keşfedilir. Oturumlar, her biri bir dizi sayfa görüntülemesinden oluşan işlemler (transaction) olarak modellenir.

Sık öge kümeleri (frequent itemsets) madenciliği, birlikte sıklıkla erişilen sayfa gruplarını bulur. Her oturum bir sepet (basket), sayfalar ise sepetteki öğeler olarak düşünülür. Örneğin minimum destek eşiği $\sigma = 0.15$ (oturumların en az %15'i) olarak belirlendiğinde, Apriori veya FP-Growth gibi algoritmalarla {AnaSayfa, Ürünler, Sepet} gibi sık birlikte erişilen sayfa kümeleri keşfedilir. Minimum destek değeri s olan bir X öge kümesi için $\text{support}(X) = \frac{|\{t \in T: X \subseteq t\}|}{|T|} \geq s$ koşulu sağlanmalıdır; burada T tüm oturumların kümesidir.

Birliktelik kuralları (association rules), sık öge kümelerinden türetilen " $X \Rightarrow Y$ " formundaki kurallardır ve bir sayfa grubuna erişen kullanıcının başka bir sayfa grubuna da erişme olasılığını ifade eder. Her kural için güven (confidence) ölçütü $\text{conf}(X \Rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)}$ olarak hesaplanır. Örneğin $\{\text{ÜrünA}\} \Rightarrow \{\text{Sepet}\}$ kuralının güveni 0.40 ise, Ürün A sayfasını görüntüleyen oturumların %40'ında sepet sayfasına da erişilmektedir. Birliktelik kurallarının web kullanım madenciliğindeki tipik uygulamaları arasında çapraz satış (cross-selling) önerileri, sayfa yerleşimi optimizasyonu ve önbelleğe alma stratejileri sayılabilir.

Ardışık örüntüler (sequential patterns), birliktelik kurallarına zaman boyutunu ekler. Bu teknikte sayfalar arasındaki erişim sırası da dikkate alınır. Bir ardışık örüntü $\langle p_1, p_2, \dots, p_k \rangle$ şeklinde, sayfaların belirli bir sırayla erişildiği dizilerdir. Minimum destek eşiğini aşan tüm sıralı diziler keşfedilir. Örneğin $\langle \text{AnaSayfa}, \text{Arama}, \text{ÜrünA}, \text{Sepet} \rangle$ ardışık örüntüsü, kullanıcıların önce arama yapıp sonra belirli bir ürünü inceleyip ardından sepete eklediği tipik bir satın alma hunisini (purchase funnel) ortaya çıkarır. Ardışık örüntü madenciliğinde GSP (Generalized Sequential Pattern) ve PrefixSpan gibi algoritmalar kullanılır. Bu örüntüler, sitenin navigasyon akışının optimize edilmesi ve kullanıcı deneyiminin iyileştirilmesi için değerli bilgiler sunar.

Kümeleme (clustering) ve sınıflandırma (classification) teknikleri, kullanıcı segmentasyonu için kullanılır. Kümeleme ile benzer gezinme davranışına sahip kullanıcı grupları otomatik olarak keşfedilir. K-ortalamlar (K-means) veya hiyerarşik kümeleme algoritmaları, oturumları sayfa görüntüleme vektörleri olarak temsil edip benzerliklerine göre gruplandırır. Örneğin bir e-ticaret sitesinde kümeleme sonucu ortaya çıkan gruplar; "hızlı alışveriş yapanlar", "araştırma yapıp çıkanlar", "fiyat karşılaştıranlar" ve "impulsif alıcılar" gibi anlamlı segmentlere karşılık gelebilir. Sınıflandırma ise önceden tanımlanmış kullanıcı kategorilerine yeni kullanıcıları atamak için kullanılır; örneğin bir kullanıcının ilk birkaç sayfa görüntülemesine bakarak satın alma olasılığının yüksek mi düşük mü olduğu tahmin edilebilir. Karar ağaçları, naif Bayes ve lojistik regresyon bu amaçla sıklıkla kullanılan sınıflandırma algoritmalarıdır.

5.10 Özet

Web kullanım madenciliği, daha kişiselleştirilmiş, kullanıcı dostu ve iş açısından optimal web hizmetleri sunmanın temel aracı haline gelmiştir. Sürecin özü, kullanıcı tıklama akışı verisini çeşitli madencilik amaçları için kullanmaktır. Veri ön işleme aşamasındaki kullanıcı tanımlama, oturumlara ayırma ve yol tamamlama gibi adımlar, analizin kalitesini doğrudan belirleyen kritik bileşenlerdir. Örüntü keşfi aşamasında uygulanan sık öge kümeleri, birliktelik kuralları, ardışık örüntüler ve kümeleme/sınıflandırma teknikleri, ham kullanım verisinden eyleme dönüştürülebilir içgörülere ulaşmayı sağlar. Geleneksel olarak e-ticaret sitelerinin organizasyonunu iyileştirmek ve kârı artırmak için kullanılan web kullanım madenciliği, günümüzde arama motorlarından içerik kişiselleştirmeye, sahtekarlık tespitinden kullanıcı deneyimi optimizasyonuna kadar çok geniş bir uygulama yelpazesine sahiptir.

6 Tavsiye Sistemleri

6.1 Uzun Kuyruk (Long Tail) Fenomeni

Geleneksel perakende satıcılar için raf alanı kısıtlı bir kaynaktır; fiziksel mağazalar yalnızca sınırlı sayıda ürünü sergileyebilir. Benzer şekilde televizyon kanalları belirli sayıda program yayımlayabilir, sinema salonları belirli sayıda film gösterebilir. İnternet ise ürünler hakkındaki bilginin neredeyse sıfır maliyetle yayılmasını sağlayarak kısıtlıktan bolluğa geçişi mümkün kılar ve Uzun Kuyruk (Long Tail) fenomenini ortaya çıkarır. Bu fenomende, yalnızca en popüler az sayıda ürün değil, popüleritesi düşük olan niş (niche) ürünler de ekonomik olarak anlamlı hale gelir; çünkü web ortamında dağıtım maliyeti neredeyse yoktur ve niş ürünler de kendilerine alıcı bulabilir. Artan seçenek sayısı, daha iyi filtreleme olan ihtiyacı doğurur ve tavsiye motorları tam da bu noktada devreye girer. Kullanıcıların tercihlerine uygun ürünleri bulmalarına yardımcı olan bu sistemler; kitap, film, müzik, haber makalesi gibi ürünlerin yanı sıra sosyal ağlarda arkadaş tavsiyesi için de kullanılır.

6.2 Formal Model

Tavsiye sistemleri formal olarak üç temel bileşenle tanımlanır. C kullanıcıların kümesini (Customers), S ise maddelerin kümesini (Items) ifade eder. Fayda fonksiyonu (utility function) $u : C \times S \rightarrow R$ her bir kullanıcı-madde çiftini bir derecelendirme değerine eşler. Burada R tam sıralı bir kümedir; örneğin 0–5 yıldız aralığı veya $[0, 1]$ reel sayı aralığı kullanılır. Tüm kullanıcı-madde çiftleri için $u(c, s)$ değerlerinin oluşturduğu matrisi fayda matrisi (utility matrix) adı verilir. Tavsiye sisteminin üç aşamalı problemi şu şekilde özetlenebilir: İlk olarak, fayda matrisindeki bilinen derecelendirmelerin toplanması; ikinci olarak, bilinmeyen derecelendirmelerin bilinenlerden yola çıkarak kestirilmesi – ki burada asıl ilgi yüksek derecelendirme tahminleridir, kullanıcının neyi sevmediğini bilmekten ziyade neyi seveceğini bilmek önemlidir; üçüncü olarak ise kestirim yöntemlerinin başarısının değerlendirilmesidir.

6.3 Veri Toplama ve Seyreklik Problemi

Derecelendirme verileri açık (explicit) veya örtük (implicit) yöntemlerle toplanır. Açık veri toplamada kullanıcılardan maddeleri doğrudan derecelendirmeleri istenir; ancak bu yöntem ölçeklenebilir değildir çünkü kullanıcıların yalnızca küçük bir kısmı derecelendirme ve yorum bırakır. Örtük veri toplamada ise kullanıcı eylemlerinden (satın alma, tıklama, izleme süresi gibi) derecelendirmeler öğrenilir; örneğin bir kullanıcının bir ürünü satın alması yüksek derecelendirme anlamına gelir, ancak düşük derecelendirmeleri örtük olarak yakalamak zordur. Fayda matrisinin en önemli problemi seyrekliliktir (sparsity): çoğu kullanıcı çoğu maddeyi derecelendirmemiştir. Buna ek olarak soğuk başlangıç (cold start) problemi yaşanır: yeni eklenen maddelerin hiç derecelendirmesi yoktur ve sisteme yeni katılan kullanıcıların hiçbir geçmiş verisi bulunmaz. Literatürde bu problemlere üç temel yaklaşım geliştirilmiştir: içerik tabanlı filtreleme, işbirlikçi filtreleme ve gizli faktör tabanlı yöntemler.

6.4 İçerik Tabanlı Filtreleme (Content-Based Filtering)

6.4.1 Ana Fikir ve Madde Profilleri

İçerik tabanlı filtrelemenin ana fikri, bir x kullanıcıya, geçmişte yüksek puan verdiği maddelere benzer yeni maddeler önermektir. Bunun için önce her maddeye ait bir profil oluşturulur. Madde profili, maddenin özelliklerini temsil eden bir vektördür; filmler için aktör, yönetmen, tür gibi özellikler, metin belgeleri için ise önemli kelimeler kullanılır. Metin tabanlı maddelerde (haber makaleleri, blog yazıları gibi) en yaygın profil oluşturma yöntemi TF-IDF (Term Frequency – Inverse Document Frequency) ağırlıklandırmasıdır. f_{ij} teriminin i teriminin j

dokümanındaki normalize edilmiş frekansı, n_i terimini içeren doküman sayısı, N ise toplam doküman sayısı olmak üzere TF-IDF skoru şu şekilde hesaplanır:

$$w_{ij} = TF_{ij} \times IDF_i \quad (48)$$

Burada $IDF_i = \log(N/n_i)$ şeklinde tanımlanır ve nadir terimlere daha yüksek ağırlık verir. TF_{ij} değeri, uzun dokümanların haksız üstünlüğünü engellemek amacıyla normalize edilir. Doküman profili, en yüksek TF-IDF skoruna sahip kelimeler ve bu skorlardan oluşur.

6.4.2 Kullanıcı Profili ve Kosinüs Benzerliği ile Tahmin

x kullanıcısının profili, derecelendirdiği maddelerin profil vektörlerinin ağırlıklı ortalaması alınarak oluşturulur. Kullanıcının 1–5 yıldız aralığında derecelendirdiği filmleri düşünelim. Sadece "Aktör" özelliğinin kullanıldığı basit bir örnekte, x kullanıcısı A aktörünün oynadığı 2 filmi (ortalama puanı 4) ve B aktörünün oynadığı 3 filmi (ortalama puanı 2.33) izlemiş olsun. Kullanıcının ortalama puanı 3 olduğuna göre, normalizasyon işlemi her puanın ortalamadan farkı alınarak yapılır: A aktörü için normalize puanlar $3 - 3 = 0$ ve $5 - 3 = +2$ olup profil ağırlığı $(0 + 2)/2 = 1$ bulunur; B aktörü için normalize puanlar $1 - 3 = -2$, $2 - 3 = -1$, $4 - 3 = +1$ olup profil ağırlığı $(-2 - 1 + 1)/3 = -2/3$ olur. Kullanıcı profili böylece oluşturulduktan sonra, yeni bir i maddesinin x kullanıcısı için tahmini faydası kosinüs benzerliği ile hesaplanır:

$$\hat{u}(x, i) = \cos(\theta) = \frac{x \cdot i}{|x| |i|} \quad (49)$$

Burada x kullanıcı profil vektörü, i madde profil vektörüdür. Kosinüs benzerliği -1 ile $+1$ arasında değer alır; $+1$ mükemmel eşleşmeyi, 0 ilişkisizliği, -1 ise tam tersliği ifade eder.

6.4.3 Avantajlar ve Dezavantajlar

İçerik tabanlı filtrelemenin önemli avantajları vardır: başka kullanıcıların verisine ihtiyaç duymaz, bu sayede benzersiz ve niş zevklere sahip kullanıcılara da tavsiye yapabilir; yeni ve popüler olmayan maddeler için ilk derecelendirici problemi (first-rater problem) yaşanmaz, çünkü tavsiye için başka kullanıcıların puanlarına gerek yoktur; ayrıca bir maddenin neden tavsiye edildiğine dair açıklama sunulabilir. Buna karşılık yöntemin belirgin dezavantajları da bulunur: görüntü, video ve müzik gibi içerikler için uygun özelliklerin çıkarılması oldukça zordur; aşırı uzmanlaşma (overspecialization) problemi yaşanır – sistem kullanıcının mevcut içerik profilinin dışına asla çıkmaz, oysa kullanıcılar birden fazla ilgi alanına sahip olabilir ve diğer kullanıcıların kalite yargılarından faydalanılamaz; yeni kullanıcılar için soğuk başlangıç problemi devam eder, çünkü henüz bir kullanıcı profili oluşturulamamıştır.

6.5 İşbirlikçi Filtreleme (Collaborative Filtering)

6.5.1 Kullanıcı-Kullanıcı (User-User) Yöntemi

Kullanıcı-kullanıcı işbirlikçi filtrelemede amaç, x kullanıcısına benzer derecelendirme davranışı gösteren diğer kullanıcıları bularak x 'in bilinmeyen puanlarını tahmin etmektir. Bunun için bir benzerlik metriğine ihtiyaç vardır. En temel metriklerden biri Jaccard benzerliğidir; iki kullanıcının ortak derecelendirdiği maddelerin, toplam derecelendirilen maddelere oranı olarak tanımlanır:

$$sim_{Jaccard}(A, B) = \frac{|r_A \cap r_B|}{|r_A \cup r_B|} \quad (50)$$

Ancak Jaccard benzerliği derecelendirme değerlerini tamamen göz ardı eder; örneğin biri her şeye 5 verirken diğeri her şeye 1 verse bile ortak maddeleri çoksa benzer görünebilirler. Bu sorunu gidermek için kosinüs benzerliği kullanılır:

$$sim_{cos}(A, B) = \cos(r_A, r_B) = \frac{r_A \cdot r_B}{|r_A| |r_B|} \quad (51)$$

Kosinüs benzerliği derecelendirme değerlerini dikkate alsa da, eksik derecelendirmeleri negatif olarak değerlendirme eğilimindedir. En iyi sonucu veren yöntem ise merkezlenmiş kosinüs (centered cosine) yani Pearson korelasyonudur. Bu yöntemde her kullanıcının derecelendirmelerinden kendi ortalama puanı çıkarılarak normalizasyon yapılır:

$$sim_{Pearson}(A, B) = \frac{\sum_{i \in I_{AB}} (r_{Ai} - \bar{r}_A)(r_{Bi} - \bar{r}_B)}{\sqrt{\sum_{i \in I_{AB}} (r_{Ai} - \bar{r}_A)^2} \sqrt{\sum_{i \in I_{AB}} (r_{Bi} - \bar{r}_B)^2}} \quad (52)$$

Burada I_{AB} , hem A hem de B kullanıcısının derecelendirdiği ortak maddelerin kümesidir; $\bar{r}_A$ ve $\bar{r}_B$ ise ilgili kullanıcıların ortalama puanlarıdır. Pearson korelasyonu, eksik puanları ortalama olarak değerlendirir ve "cimri derecelendirici" (tough rater) ile "cömert derecelendirici" (easy rater) arasındaki farkı dengeler.

Benzer kullanıcılar belirlendikten sonra, x kullanıcısının i maddesi için tahmini puanı, en benzer k kullanıcının (komşuların) ağırlıklı ortalaması ile hesaplanır. N komşu kümesi ve $s_{xy} = sim(x, y)$ olmak üzere:

$$\hat{r}_{xi} = \frac{\sum_{y \in N} s_{xy} \cdot r_{yi}}{\sum_{y \in N} |s_{xy}|} \quad (53)$$

6.5.2 Madde-Madde (Item-Item) Yöntemi

Madde-madde işbirlikçi filtreleme, kullanıcı-kullanıcı yaklaşımının dualidir. Burada i maddesine benzer diğer maddeler bulunur ve x kullanıcısının i maddesi için tahmini puanı, x 'in benzer maddelere verdiği puanlar üzerinden hesaplanır. Pratikte madde-madde yöntemi birçok kullanım durumunda kullanıcı-kullanıcı yönteminden daha iyi performans gösterir; çünkü maddeler kullanıcılara göre "daha basittir" – maddeler belirli türlere (genre) aittir ve özellikleri zaman içinde sabit kalır, oysa kullanıcıların zevkleri çok çeşitli ve değişkendir. Madde benzerliği, kullanıcı benzerliğine kıyasla daha anlamlı ve kararlıdır.

Madde-madde yönteminin tam sayısal bir örnekle açıklanması konunun anlaşılması için önemlidir. 6 kullanıcı ve 12 filmden oluşan bir fayda matrisi düşünelim. Film 1 için kullanıcı puanları sırasıyla şöyledir: Kullanıcı 1: 1, Kullanıcı 2: bilinmiyor, Kullanıcı 3: 3, Kullanıcı 4: 5, Kullanıcı 5: 5, Kullanıcı 6: 4. Amacımız Kullanıcı 5'in Film 1 için tahmini puanını ($\hat{r}_{51}$) hesaplamaktır.

Adım adım ilerleyelim. İlk olarak Film 1'in ortalama puanı hesaplanır (yalnızca bilinen puanlar üzerinden):

$$\bar{r}_1 = \frac{1 + 3 + 5 + 5 + 4}{5} = \frac{18}{5} = 3.6 \quad (54)$$

İkinci adımda, Film 1'in her bir kullanıcı için normalize edilmiş puan vektörü oluşturulur (her puandan $\bar{r}_1 = 3.6$ çıkarılır, bilinmeyen değerlere 0 atanır): $[-2.6, 0, -0.6, 0, 0, 1.4, 0, 0, 1.4, 0, 0.4, 0]$. Aynı normalizasyon işlemi diğer tüm filmler için de yapılır. Üçüncü adımda, Film 1 ile diğer filmler arasındaki Pearson korelasyon (yani normalize vektörlerin kosinüs) benzerlikleri hesaplanır. Hesaplamalar sonucunda Film 1 ile en yüksek pozitif korelasyona sahip iki film belirlenir: Film 3 ile benzerlik $s_{13} = 0.41$ ve Film 6 ile benzerlik $s_{16} = 0.59$. Dördüncü ve son adımda, Kullanıcı 5'in bu benzer filmlere verdiği puanlar kullanılarak ağırlıklı ortalama ile tahmin yapılır. Kullanıcı 5 Film 3'e 2 puan, Film 6'ya ise 3 puan vermiştir:

$$\hat{r}_{51} = \frac{s_{13} \cdot r_{53} + s_{16} \cdot r_{56}}{s_{13} + s_{16}} = \frac{0.41 \times 2 + 0.59 \times 3}{0.41 + 0.59} = \frac{0.82 + 1.77}{1.00} = 2.59 \quad (55)$$

Buna göre Kullanıcı 5'in Film 1 için tahmini puanı yaklaşık 2.6 olarak bulunur. Bu örnek, madde-madde işbirlikçi filtrelemenin nasıl çalıştığını adım adım göstermektedir: ortalama ile normalizasyon, Pearson korelasyonu ile benzerlik hesabı ve ağırlıklı ortalama ile nihai tahmin.

6.6 Değerlendirme Metrikleri

6.6.1 RMSE (Root Mean Square Error)

Tavsiye sistemlerinin başarısını ölçmek için en yaygın kullanılan metriklerden biri Kök Ortalama Kare Hatasıdır (RMSE). Değerlendirme için, bilinen derecelendirmelerin bir kısmı test kümesi olarak ayrılır (T) ve sistemin bu gizlenmiş puanlar üzerindeki tahmin performansı ölçülür. r_{xi} tahmin edilen puanı, r_{xi}^* gerçek puanı ve $N = |T|$ test kümesindeki toplam değerlendirme sayısını göstermek üzere RMSE formülü şöyledir:

$$RMSE = \sqrt{\frac{1}{N} \sum_{(x,i) \in T} (r_{xi} - r_{xi}^*)^2} \quad (56)$$

RMSE, tahmin hatasının büyüklüğünü doğrudan cezalandırır ve hataların karesini aldığı için büyük hatalara karşı özellikle hassastır. Düşük RMSE değeri, sistemin daha doğru tahminler yaptığını gösterir.

Ancak RMSE'nin önemli zayıflıkları vardır. Yalnızca doğruluk (accuracy) odaklı dar bir bakış açısı sunar ve tahmin çeşitliliğini (diversity), tahmin bağlamını (context) ve tahminlerin sıralamasını (order) göz ardı eder. Pratikte asıl önemli olan yüksek puanları doğru tahmin etmektir; bir kullanıcının neyi seveceğini bilmek, neyi sevmeyeceğini bilmekten çok daha değerlidir. RMSE ise düşük puanlardaki hataları da aynı ağırlıkla cezalandırır ve yüksek puanlarda başarılı olup düşük puanlarda kötü performans gösteren bir yöntemi haksız şekilde cezalandırabilir.

6.6.2 Precision at Top-K

RMSE'nin bu sınırlamalarını aşmak için Precision at Top-K (İlk K'da Hassasiyet) metriği geliştirilmiştir. Bu metrik yalnızca yüksek puanlı tahminlerle ilgilenir ve şu soruyu sorar: sistemin kullanıcıya önerdiği en iyi K maddenin ne kadarı, kullanıcının gerçekten yüksek puan verdiği (test kümesinde gizlenmiş) maddeler arasındadır? Formel olarak, her bir x kullanıcısı için sistemin tahmin ettiği en yüksek puanlı K madde kümesi $TopK(x)$ ve kullanıcının test kümesinde 4 ve üzeri (veya belirli bir eşik değerin üzerinde) puan verdiği maddelerin kümesi $High(x)$ olmak üzere:

$$Precision@K = \frac{1}{|U|} \sum_{x \in U} \frac{|TopK(x) \cap High(x)|}{K} \quad (57)$$

Burada U test kümesindeki kullanıcıların kümesidir. Precision at Top-K metriğinin avantajı, yalnızca sistemin en kendinden emin olduğu ve kullanıcı için en değerli olan yüksek puanlı tahminlere odaklanmasıdır. Örneğin bir kullanıcının test kümesinde 5, 5, 4, 2, 1 puan verdiği 5 gizli film olsun ve $High(x)$ kümesi eşik 4 için $\{5, 5, 4\}$ olarak belirlensin. Sistem $K = 5$ için en yüksek tahmin ettiği 5 filmi önersin ve bunlardan 2 tanesi gerçekten kullanıcının $High(x)$ kümesinde yer alsın. Bu durumda $Precision@5 = 2/5 = 0.4$ olur. Bu metrik, RMSE'nin aksine düşük puanlardaki tahmin hatalarından etkilenmez ve tavsiye sisteminin asıl amacı olan "kullanıcının seveceği maddeleri bulma" hedefine daha uygun bir değerlendirme sunar.

7 Formüller

7.1 Birliktelik Kuralları (Association Rules)

Destek (support), bir ürün kümesinin tüm işlemler içinde görülme olasılığını ifade eder ve hem X hem de Y ürünlerini birlikte içeren işlemlerin oranı olarak hesaplanır.

$$\text{destek}(X \cup Y) = \frac{|\{t \in T : X \cup Y \subseteq t\}|}{|T|} = P(X \cup Y) \quad (58)$$

Güven (confidence), X ürünlerini içeren işlemler arasında Y ürünlerini de içerenlerin koşullu olasılığıdır ve destek değerleri üzerinden hesaplanır.

$$\text{güven}(X \rightarrow Y) = \frac{\text{destek}(X \cup Y)}{\text{destek}(X)} = P(Y | X) \quad (59)$$

Sık ürün kümelerinden kural üretimi aşamasında, bir X sık ürün kümesinin boş olmayan öz alt kümeleri $A \subset X$ için $B = X - A$ olmak üzere güven değeri aşağıdaki gibi hesaplanır ve minconf eşliğini geçen kurallar kabul edilir.

$$\text{güven}(A \rightarrow B) = \frac{\text{destek}(X)}{\text{destek}(A)} \quad (60)$$

Apriori algoritmasının ikili sayımında kullanılan üçgensel matris yönteminde, $\{i, j\}$ çiftinin ($i < j$) tek boyutlu dizideki konumu aşağıdaki formülle hesaplanır; burada n toplam ürün sayısıdır.

$$\text{konum}(i, j) = (i - 1) \left(n - \frac{i}{2} \right) + j - i \quad (61)$$

Sık öge kümesi madencilğinde bir X öge kümesinin destek değeri, X 'i içeren oturum sayısının toplam oturum sayısına oranıdır ve minimum destek eşiği s ile karşılaştırılır.

$$\text{support}(X) = \frac{|\{t \in T : X \subseteq t\}|}{|T|} \geq s \quad (62)$$

7.2 Bilgi Erişimi (Information Retrieval)

Ters doküman frekansı (IDF), bir terimin koleksiyon genelinde ne kadar ayırt edici olduğunu ölçer; N toplam doküman sayısı, df_i ise t_i terimini içeren doküman sayısıdır.

$$\text{IDF}(t_i) = \log \left(\frac{N}{df_i} \right) \quad (63)$$

TF-IDF birleşik ağırlığı, terimin doküman içi frekansı (TF) ile koleksiyon geneli nadirliğinin (IDF) çarpımıdır; f_{ij} terim frekansını gösterir.

$$w_{ij} = \text{TF}(t_i, d_j) \times \text{IDF}(t_i) = f_{ij} \cdot \log \left(\frac{N}{df_i} \right) \quad (64)$$

Vektör uzay modelinde bir doküman ile sorgu arasındaki benzerlik, iki vektör arasındaki kosinüs benzerliği ile ölçülür; w_{iq} sorgu terim ağırlığını, w_{ij} doküman terim ağırlığını gösterir.

$$\cos(q, d_j) = \frac{q \cdot d_j}{\|q\| \cdot \|d_j\|} = \frac{\sum_{i=1}^{|V|} w_{iq} \cdot w_{ij}}{\sqrt{\sum_{i=1}^{|V|} w_{iq}^2} \cdot \sqrt{\sum_{i=1}^{|V|} w_{ij}^2}} \quad (65)$$

Okapi BM25, TF-IDF ailesinin en başarılı varyantlarından biri olup TF doygunluğu ve doküman uzunluk normalizasyonu sağlar; k_1 ve b ayarlanabilir parametrelerdir.

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgl}}\right)} \quad (66)$$

BM25'te kullanılan IDF bileşeni, sıfıra bölünmeyi engellemek için yumuşatma (smoothing) terimleri içerecek şekilde uyarlanmıştır.

$$\text{IDF}_{\text{BM25}}(t) = \log \left(\frac{N - \text{df}_t + 0.5}{\text{df}_t + 0.5} \right) \quad (67)$$

Rocchio ilgililik geri bildirimi yöntemi, kullanıcının ilgili (D_r) ve ilgisiz (D_{ir}) olarak işaretlediği dokümanları kullanarak orijinal sorgu vektörünü q_0 genişletir; α , β ve γ kontrol parametreleridir.

$$q_m = \alpha \cdot q_0 + \beta \cdot \frac{1}{|D_r|} \sum_{d_j \in D_r} d_j - \gamma \cdot \frac{1}{|D_{ir}|} \sum_{d_j \in D_{ir}} d_j \quad (68)$$

Kesinlik (precision), sistemin getirdiği dokümanlar arasında gerçekten ilgili olanların oranını ölçer; R ilgili dokümanlar kümesi, S ise sistemin döndürdüğü dokümanlar kümesidir.

$$\text{Kesinlik (Precision)} = \frac{|R \cap S|}{|S|} \quad (69)$$

Geri çağırma (recall), koleksiyondaki tüm ilgili dokümanların ne kadarının sistem tarafından getirildiğini ölçer.

$$\text{Geri Çağırma (Recall)} = \frac{|R \cap S|}{|R|} \quad (70)$$

F-skoru, kesinlik ve geri çağırmanın harmonik ortalamasıdır ve sistem karşılaştırmalarında tek bir dengeli sayısal değer sunar.

$$F = \frac{2 \cdot \text{Kesinlik} \cdot \text{Geri Çağırma}}{\text{Kesinlik} + \text{Geri Çağırma}} \quad (71)$$

7.3 Bağlantı Analizi ve Merkezilik Ölçütleri (Link Analysis & Centrality)

Freeman merkezileşme formülü, bir ağdaki derece merkeziliği skorlarının dağılımındaki asimetriyi ölçer; $C_D(n^*)$ ağdaki en yüksek derece merkeziliğidir.

$$C_D = \frac{\sum_{i=1}^n [C_D(n^*) - C_D(i)]}{\max \sum_{i=1}^n [C_D(n^*) - C_D(i)]} \quad (72)$$

Yakınlık merkeziliği, bir düğümün ağdaki diğer tüm düğümlere olan ortalama en kısa yol uzaklığının tersidir; $d(i, j)$, i ile j arasındaki en kısa yol uzaklığıdır.

$$C_C(i) = \frac{1}{\sum_{j \neq i} d(i, j)} \quad (73)$$

Normalleştirilmiş yakınlık merkeziliği, ham değer ($N - 1$) ile çarpılmasıyla $[0, 1]$ aralığına ölçeklenir.

$$C'_C(i) = \frac{N - 1}{\sum_{j \neq i} d(i, j)} \quad (74)$$

Arasındalık merkeziliği, bir düğümün diğer düğüm çiftleri arasındaki en kısa yollar üzerinde bulunma sıklığını ölçer; p_{jk} toplam en kısa yol sayısı, $p_{jk}(i)$ ise bunlardan i 'den geçenlerin sayısıdır.

$$C_B(i) = \sum_{j < k} \frac{p_{jk}(i)}{p_{jk}} \quad (75)$$

Yakınlık prestiji, bir düğüme ulaşabilen tüm aktörleri ve ortalama uzaklıklarını dikkate alan rafine bir prestij ölçüsüdür; I_i , i düğüme ulaşabilen aktörlerin kümesidir.

$$P_P(i) = \frac{|I_i|/(N-1)}{\sum_{j \in I_i} d(j, i)/|I_i|} \quad (76)$$

Sıra prestiji, gelen bağlantıların ağırlıklı doğrusal birleşimi olarak tanımlanır ve PageRank'in temelini oluşturur; A bitişiklik matrisidir.

$$P(i) = \sum_j A_{ji} P(j) \quad (77)$$

PageRank'in temel akış denklemi, bir sayfanın önem skorunun kendisine bağlantı veren sayfaların skorlarının giden bağlantı sayılarına bölünmüş toplamı olduğunu ifade eder.

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i} \quad (78)$$

PageRank akış denklemlerinin matris formülasyonu, $M_{ji} = 1/d_i$ (eğer $i \rightarrow j$ bağlantısı varsa) olarak tanımlanan sütun-stokastik matris ile ifade edilir.

$$r = M \cdot r \quad (79)$$

Google'ın rastgele atlama modeli ile genişletilmiş PageRank formülü, β sönümleme faktörü (tipik olarak 0.85) ile bağlantı tabanlı oy aktarımını ve teleport bileşenini birleştirir.

$$r_j = \beta \sum_{i \rightarrow j} \frac{r_i}{d_i} + (1 - \beta) \frac{1}{n} \quad (80)$$

Google PageRank'in vektör formundaki ifadesi; e tüm elemanları 1 olan n boyutlu sütun vektörüdür.

$$r = \beta M r + (1 - \beta) \frac{1}{n} e \quad (81)$$

Google matrisi A , sütun-stokastik, indirgenemez ve periyodik olmayan bir geçiş matrisidir; ee^T tüm elemanları 1 olan $n \times n$ boyutlu kare matristir.

$$A = \beta M + (1 - \beta) \frac{1}{n} ee^T \quad (82)$$

PageRank vektörü, Google matrisinin 1 özdeğerine karşılık gelen esas özvektörüdür.

$$r = A \cdot r \quad (83)$$

HITS algoritmasında bir sayfanın otorite skoru, kendisine bağlantı veren göbek sayfalarının skorlarının toplamıdır.

$$a(i) = \sum_{(j,i) \in E} h(j) \quad (84)$$

HITS algoritmasında bir sayfanın göbek skoru, bağlantı verdiği otorite sayfalarının skorlarının toplamıdır.

$$h(i) = \sum_{(i,j) \in E} a(j) \quad (85)$$

HITS denklemlerinin matris formunda ifadesi; L bitişiklik matrisi, a otorite vektörü, h göbek vektörüdür.

$$a = L^T h \quad (86)$$

$$h = La \quad (87)$$

HITS'te otorite ve göbek vektörleri sırasıyla $L^T L$ ve LL^T matrislerinin esas özvektörleridir.

$$a = (L^T L)a \quad (88)$$

$$h = (LL^T)h \quad (89)$$

Eş-atıf (co-citation) matrisi C , iki sayfanın aynı üçüncü sayfalar tarafından birlikte referans gösterilme sıklığını ölçer; $C = L^T L$ 'dir.

$$C = L^T L, \quad C_{ij} = \sum_{k=1}^n L_{ki} L_{kj} \quad (90)$$

Bibliyografik eşleşme (bibliographic coupling) matrisi B , iki sayfanın aynı üçüncü sayfalara birlikte atıf yapma sıklığını ölçer; $B = LL^T$ 'dir.

$$B = LL^T, \quad B_{ij} = \sum_{k=1}^n L_{ik} L_{jk} \quad (91)$$

7.4 Bloom Filtreleri (Bloom Filters)

Bloom filtresinde t bitlik dizide d adet dart (hash) atıldığında, belirli bir bitin 0 olarak kalma olasılığı aşağıdaki gibidir.

$$P(\text{bit 0 kalır}) = \left(1 - \frac{1}{t}\right)^d \quad (92)$$

t büyük olduğunda, bit kalma olasılığı üstel fonksiyona yakınsar ve $e^{-d/t}$ ile ifade edilir.

$$\left(1 - \frac{1}{t}\right)^d = \left[\left(1 - \frac{1}{t}\right)^t\right]^{d/t} \approx e^{-d/t} \quad (93)$$

Yanlış pozitif olasılığı, daha önce eklenmemiş bir eleman için k bağımsız hash fonksiyonunun tamamının hâlihazırda 1 olan bitlere işaret etmesi durumunda gerçekleşir; m eklenen eleman sayısıdır.

$$P(\text{FP}) = \left(1 - e^{-d/t}\right)^k = \left(1 - e^{-km/t}\right)^k \quad (94)$$

7.5 Tavsiye Sistemleri (Recommender Systems)

İçerik tabanlı filtrelemede TF-IDF ağırlıklandırması, terim frekansı TF_{ij} ile ters doküman frekansı IDF_i 'nin çarpımıdır; n_i terimi içeren doküman sayısıdır.

$$w_{ij} = TF_{ij} \times IDF_i, \quad IDF_i = \log \left(\frac{N}{n_i} \right) \quad (95)$$

Kullanıcı profili ile madde profili arasındaki kosinüs benzerliği, içerik tabanlı filtrelemede tahmini faydayı hesaplamak için kullanılır.

$$\hat{u}(x, i) = \cos(\theta) = \frac{x \cdot i}{|x| |i|} \quad (96)$$

Jaccard benzerliği, iki kullanıcının ortak derecelendirdiği maddelerin toplam derecelendirilen maddelere oranıdır; derecelendirme değerlerini dikkate almaz.

$$sim_{Jaccard}(A, B) = \frac{|r_A \cap r_B|}{|r_A \cup r_B|} \quad (97)$$

İki kullanıcı arasındaki kosinüs benzerliği, kullanıcı derecelendirme vektörlerinin iç çarpımının uzunlukları çarpımına bölünmesiyle hesaplanır.

$$sim_{cos}(A, B) = \cos(r_A, r_B) = \frac{r_A \cdot r_B}{|r_A| |r_B|} \quad (98)$$

Pearson korelasyonu (merkezlenmiş kosinüs), her kullanıcının derecelendirmelerinden kendi ortalaması çıkarılarak hesaplanır ve derecelendirme eğilim farklarını dengeler; I_{AB} ortak maddeler kümesidir.

$$sim_{Pearson}(A, B) = \frac{\sum_{i \in I_{AB}} (r_{Ai} - \bar{r}_A)(r_{Bi} - \bar{r}_B)}{\sqrt{\sum_{i \in I_{AB}} (r_{Ai} - \bar{r}_A)^2} \sqrt{\sum_{i \in I_{AB}} (r_{Bi} - \bar{r}_B)^2}} \quad (99)$$

Kullanıcı-kullanıcı işbirlikçi filtrelemede tahmini puan, en benzer k komşunun ağırlıklı ortalaması ile hesaplanır; N komşu kümesi, s_{xy} benzerlik skorudur.

$$\hat{r}_{xi} = \frac{\sum_{y \in N} s_{xy} \cdot r_{yi}}{\sum_{y \in N} |s_{xy}|} \quad (100)$$

Kök ortalama kare hatası (RMSE), tavsiye sistemlerinin tahmin doğruluğunu ölçen temel metriktir; r_{xi} tahmini puan, r_{xi}^* gerçek puan, T test kümesidir.

$$RMSE = \sqrt{\frac{1}{N} \sum_{(x,i) \in T} (r_{xi} - r_{xi}^*)^2} \quad (101)$$

İlk K 'da hassasiyet (Precision@K), sistemin önerdiği en iyi K maddenin ne kadarının kullanıcının gerçekten yüksek puan verdiği maddeler arasında olduğunu ölçer; U test kullanıcılarının kümesidir.

$$Precision@K = \frac{1}{|U|} \sum_{x \in U} \frac{|TopK(x) \cap High(x)|}{K} \quad (102)$$